Maximum Defective Biclique Search in Large Bipartite Graphs

Donghang Cui[†], Rong-Hua Li[†], Qiangqiang Dai[†], Hongchao Qin[†], Guoren Wang[†]
[†]Beijing Institute of Technology, China
{cuidonghang@bit.edu.cn,lironghuabit@126.com,qiangd66@gmail.com,qhc.neu@gmail.com,wanggrbit@126.com}

Abstract

The problem of identifying the maximum edge biclique in bipartite graphs has attracted considerable attention in bipartite graph analysis, with numerous real-world applications such as fraud detection, community detection, and online recommendation systems. However, real-world graphs may contain noise or incomplete information, leading to overly restrictive conditions when employing the biclique model. To mitigate this, we focus on a new relaxed subgraph model, called the k-defective biclique, which allows for up to k missing edges compared to the biclique model. We investigate the problem of finding the maximum edge k-defective biclique in a bipartite graph, and prove that the problem is NP-hard. To tackle this computation challenge, we propose a novel algorithm based on a new branch-and-bound framework, which achieves a worst-case time complexity of $O(m\alpha_k^n)$, where $\alpha_k < 2$. We further enhance this framework by incorporating a novel pivoting technique, reducing the worst-case time complexity to $O(m\beta_k^n)$, where $\beta_k < \alpha_k$. To improve the efficiency, we develop a series of optimization techniques, including graph reduction methods, novel upper bounds, and a heuristic approach. Extensive experiments on 10 large real-world datasets validate the efficiency and effectiveness of the proposed approaches. The results indicate that our algorithms consistently outperform state-of-the-art algorithms, offering up to 1000× speedups across various parameter settings.

PVLDB Reference Format:
Donghang Cui[†], Rong-Hua Li[†], Qiangqiang Dai[†], Hongchao Qin[†], Guoren Wang[†] . Maximum Defective Biclique Search in Large Bipartite Graphs. PVLDB, 19(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

1 Introduction

Bipartite graph is a commonly encountered graph structure consisting of two disjoint vertex sets and an edge set, where each edge connects vertices from the two different sets. In recent decades, bipartite graphs have been utilized as a fundamental tool for modeling various binary relationships, such as user-product interactions in e-commerce [32], authorship connections between authors and publications [31], and associations between genes and phenotypes in bioinformatics [17, 42]. Mining cohesive subgraphs from bipartite graphs has proven to be critically important across diverse domains [15, 50]. For instance, identifying cohesive subgraphs in reviewer-product graphs facilitates the identification of fraudulent reviews or camouflage attacks [24].

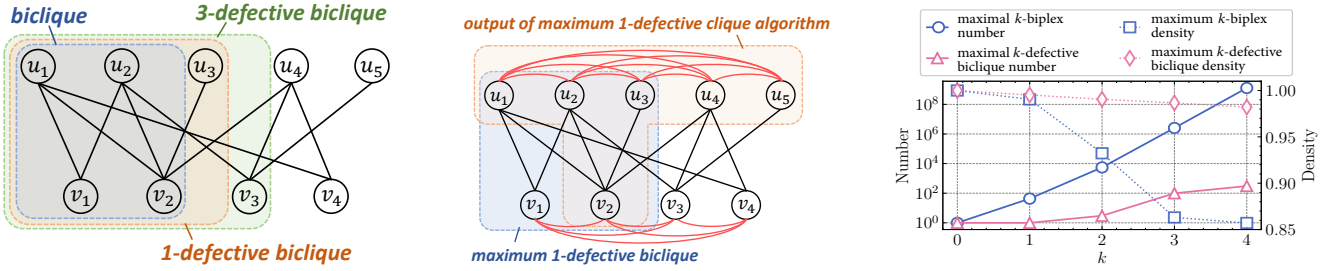
The biclique, which connects all vertices on both sides, serves as a fundamental cohesive subgraph structure [33, 38, 51] due to its wide applications [3, 6, 29, 30]. Extensive methods for maximum biclique search have been developed in recent years [3, 15, 18, 22, 35, 37]. However, real-world graphs often contain noisy or incomplete information [34], making the biclique model overly restrictive for capturing cohesive subgraphs on such a kind of bipartite graphs. More relaxed biclique models offer a promising alternative to address this challenge. The k-biplex model, which allows each vertex to have at most k non-neighbors, has recently attracted significant attention [15, 55, 56]. However, due to its excessive structural relaxation, a slight increase in k can lead to: (1) a sharp rise in quantity, potentially reducing the mining efficiency; and (2) a significant decrease in density, undermining the capability to represent cohesive communities.

Inspired by the widely studied k-defective clique in traditional graphs [12, 13, 54], we introduce a novel cohesive subgraph model for bipartite graphs, named k-defective biclique. Specifically, a k-defective biclique is a subgraph of a bipartite graph containing at most k missing edges. Fig. 1(a) illustrates an example graph showcasing two k-defective bicliques, where the 1-defective biclique has one missing edge (u_3, v_1), and the 3-defective biclique has three missing edges (u_1, v_3), (u_3, v_1) and (u_3, v_3).

The k-defective biclique offers greater flexibility over biclique, as its density can be controlled by adjusting k to suit specific requirements. When $k = 0$, the k-defective biclique degenerates to the biclique, thereby illustrating its generalization capability. In comparison to the k-biplex, k-defective biclique offers a more fine-grained relaxation of the biclique due to its stricter retention of edges. As illustrated in Fig. 1(c), the k-defective biclique exhibits much closer quantity and density to the biclique compared to k-biplex, especially for larger k. Here, we refer to a k-defective biclique as "maximal" if it cannot be contained within any other k-defective biclique (or k-biplex), and as "maximum" when it is the maximal k-defective biclique (or k-biplex) with the largest number of edges. In this paper, we focus mainly on the problem of identifying a maximum k-defective biclique. We prove that the maximum k-defective biclique search problem is NP-hard, indicating that no polynomial-time algorithms exist for solving this problem unless NP=P.

Notably, many studies have addressed the maximum k-defective clique search problem on traditional graphs [8, 9, 12, 13, 20, 28], which mainly focus on designing branch-and-bound algorithms to identify the maximum k-defective clique. A straightforward idea to applying these algorithms to the maximum k-defective biclique search is to fully connect the vertices on each side of the bipartite graph and then use these algorithms to find a maximum k-defective clique. However, such an approach may lead to suboptimal results. For instance, in Fig. 1(b), the maximum 1-defective biclique has 5 edges whereas the output of maximum 1-defective clique algorithms yields only 4 edges. In addition, several studies have proposed algorithms for the maximum biclique search problem [22, 37, 47, 51]. However, due to the presence of missing edges, identifying a maximum k-defective biclique through maximum biclique search is non-trivial, and adapting these algorithms to the maximum k-defective biclique search problem remains a challenging task.

Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX



(a) Example graph showcasing a biclique and two k-defective cliques

(b) Example of incorrect output from k-defective clique algorithms

(c) Comparison of quantity and density between k-biplex and k-defective biclique on *Amazon* dataset with side size ≥ 14

Figure 1: Illustrations of k-defective biclique: examples and comparisons

Contributions. To address these issues, we comprehensively study the maximum k-defective biclique search problem, and make the following contributions:

A new cohesive subgraph model. We propose the k-defective biclique, a new cohesive subgraph model in bipartite graph that allows up to k missing edges, and mathematically prove that the maximum k-defective biclique search problem is NP-hard.

Two Novel Search Algorithms. We propose two novel maximum k-defective biclique search algorithms with nontrivial worst-case time complexity guarantees. The first algorithm utilizes a branch-and-bound technique combined with a newly-developed binary branching strategy, which has a time complexity of $O(m\alpha_k^n)$, where n and m represent the number of vertices and edges of the bipartite graph respectively, and α_k is a positive real number strictly less than 2 (e.g., for $k = 1, 2, 3$, $\alpha_k = 1.911, 1.979$, and 1.995, respectively). The second algorithm employs an innovative pivoting-based branching strategy that effectively integrates with our binary branching strategy, which achieves a lower time complexity of $O(m\beta_k^n)$, where β_k is a positive real number strictly less than α_k. (e.g., when $k = 1, 2, 3$, $\beta_k = 1.717, 1.856$, and 1.931, respectively).

Nontrivial optimization techniques. We develop a series of nontrivial optimization techniques to improve the efficiency of our proposed algorithms, including graph reduction techniques, novel upper-bounding techniques, a heuristic approach, and parallelization techniques. Specifically, We propose graph reduction techniques to reduce the size of input graph based on constraints, such as the number of common neighbors. We devise tight vertex and edge upper bounds to minimize unnecessary branching, and design an efficient algorithm to compute the proposed upper bounds in linear time. Additionally, we propose a heuristic approach by incrementally expanding subsets in a specific order to obtain an initial large solution. We also parallelize our algorithms to further enhance their practical efficiency.

Comprehensive experimental studies. We conduct extensive experiments to evaluate the efficiency, effectiveness, and scalability of our algorithms on 11 large real-world graphs. The experimental results show that our algorithms consistently outperform the state-of-the-art baselines across all parameter settings. In particular, our algorithm achieves improvements of up to three orders of magnitude on most datasets. For instance, on the *LKML* (9M vertices, 10M edges) dataset with $k = 3$, our algorithm takes 10 seconds while the best baseline takes about 2.5 hours. Additionally, we perform case studies on a fraud detection task and an online recommendation task to show the high effectiveness of our solutions in real-world applications. The source code is available at https://github.com/pluv1a/MaximumDefectiveBiclique.

2 Preliminaries

Let $G = (U, V, E)$ be an undirected bipartite graph, where U and V are two disjoint sets of vertices, and $E \subseteq U \times V$ is the set of edges. Denote by $n = |U| + |V|$ and $m = |E|$ the total number of vertices and edges in G, respectively. Denote by $S = (U_S, V_S)$ a vertex subset of G, where $U_S \subseteq U$ and $V_S \subseteq V$. Let $v \in S$ indicate that $v \in U_S \cup V_S$. Let $G(S) = (U_S, V_S, E_S)$ be the subgraph of G induced by S, where $E_S = \{(u, v) \mid u \in U_S \wedge v \in V_S \wedge (u, v) \in E\}$. For $u \in U$, denote by $N(u) = \{v \in V \mid (u, v) \in E\}$ and $\overline{N}(u) = \{v \in V \mid (u, v) \notin E\}$ the set of neighbors and non-neighbors of u in G, respectively. Similar definitions apply for the vertices in U. Let $d(u) = |N(u)|$ and $\overline{d}(u) = |\overline{N}(u)|$ be the degree and non-degree of u in G, respectively. For a vertex subset (or a subgraph) S of G, let $N_S(u) = N(u) \cap (U_S \cup V_S)$ and $\overline{N}_S(u) = \{v \in S \setminus N(u) \mid u \text{ and } v \text{ are on opposite sides}\}$ be the set of u's neighbors and non-neighbors in S. respectively. Let $d_S(u) = |N_S(u)|$ and $\overline{d}_S(u) = |\overline{N}_S(u)|$ be the degree and non-degree of u in S, respectively. For a bipartite graph $G = (U, V, E)$, we refer to $(u, v) \in U \times V$ as a non-edge of G if $(u, v) \notin E$. We denote $\overline{E}_S$ the set of non-edges in $G(S)$, i.e., $\overline{E}_S = U_S \times V_S \setminus E_S$. The formal definition of k-defective biclique is given below.

Definition 1 (k-defective biclique). Given a bipartite graph $G = (U, V, E)$, a k-defective biclique $D = (U_D, V_D, E_D)$ of G is a subgraph of G that has at most k non-edges, i.e., $|U_D \times V_D \setminus E_D| \leq k$.

The k-defective biclique corresponds to the biclique if $k = 0$, which indicates that k-defective biclique represents a more general structure. If a k-defective biclique of G contains the most number of edges among all k-defective bicliques in G, we refer to it as a maximum k-defective biclique in G. To facilitate the illustration, we denote the maximum k-defective biclique as k-MDB.

In this paper, we focus on the problem of searching for a k-MDB in a bipartite graph. However, we observe that a k-MDBs in many real-world graphs may exhibit a skewed structure, similar to that of maximum bicliques, where one side has a large number of vertices while the other side may contain only a few (or even one) vertices. Such k-MDBs often hold limited practical values in network analysis. To address this issue, we define a size threshold θ, representing the minimum number of vertices required on both sides of the k-MDB. Furthermore, we impose the condition $\theta > k$ to ensure that the k-MDB is connected, as demonstrated in the following lemma.

Lemma 1. Given a bipartite graph $G = (U, V, E)$ and a k-defective biclique $D = (U_D, V_D, E_D)$ of G, if $|U_D| > k$ and $|V_D| > k$, we establish that D is a connected subgraph in G.

Proof. Assume that a k-defective biclique D is not connected when $|U_D| > k$ and $|V_D| > k$. In this case, D can be decomposed into two nonempty subgraphs $D_1 = (U_{D_1}, V_{D_1}, E_{D_1})$ and $D_2 = (U_{D_2}, V_{D_2}, E_{D_2})$, with the results of $E_{D_1} \cup E_{D_2} = E_D$.The number of non-edges between D_1 and D_2 is given by $|U_{D_1}| \cdot |V_{D_2}| + |U_{D_2}| \cdot |V_{D_1}|$, which exceeds $|U_D|$ (or $|V_D|$) and is thus larger than k. This contradicts the assumption that D is a k-defective biclique. Therefore, the proof is complete. □

Based on Lemma 1, we formally define the problem addressed in this paper as follows.

Problem statement (MDB search problem)**.** Given a bipartite graph $G = (U, V, E)$ and two integers k and θ with $\theta > k$, the MDB search problem aims to find a connected k-MDB $D = (U_D, V_D, E_D)$ of G satisfying $|U_D| \geq \theta$ and $|V_D| \geq \theta$.

Theorem 2. The MDB search problem is NP-hard.

Proof. We establish the NP-hardness of the MDB search problem through a polynomial-time reduction from the classical NP-complete maximum clique search problem. Our reduction approach extends the methodology presented in [41]. We assume without loss of generality that $\theta = 0$ and $k \geq 1$. The decision versions of two problems are formally defined as:

- *CLIQUE*: For an undirected graph $G = (V, E)$ and a positive integer a, does G contain a clique with at least a vertices?
- *MDB*: For a bipartite graph $G' = (U', V', E')$ and a positive integer a', does G' contains a k-defective biclique with at least a' edges?

Let $G_0 = (V_0, E_0)$ and a_0 be an input instance of *CLIQUE*. We firstly construct a modified *CLIQUE* instance with $G = (V, E)$ and a as follows:

$$V = V_0 \cup X \cup Y, \quad E = E_0 \cup (V \times X), \quad a = \frac{1}{2}|V|$$

where X and Y are two sets of new vertices with $|X| = \max\{5k, |V_0| - 2a_0 + k\}$ and $|Y| = 2a_0 + |X| - |V_0|$. This construction can be performed in polynomial time and satisfies that (1) $|X| \geq 5k$ and $|Y| \geq k$, and (2) G_0 has a clique $C = (V_C, E_C)$ of size a_0 iff G has a clique $G(V_C \cup X)$ of size $a_0 + |X| = \frac{1}{2}(|Y| - |X| + |V_0|) + |X| = a$. Next, we construct an *MDB* instance with $G' = (U', V', E')$ and a' as follows:

$$U' = V, \quad V' = E \cup W_1 \cup W_2, \quad a' = a^2(a - \frac{3}{2}) - k$$

where W_1 and W_2 are two sets of new vertices with $|W_1| = k$ and $|W_2| = \frac{1}{2}a^2 - a - k$. Note that $|W_2| > 0$ as $a = \frac{1}{2}|V| \geq 3k$ and $k \geq 1$. Let $W_1 = \{w_1, w_2, \ldots, w_k\}$, and select any k vertices $\{v_1, v_2, \ldots, v_k\} \subseteq Y$. Then the edge set E' is defined as:

$$E' = \{(v, e) \mid v \in V, e \in E, v \notin e\} \cup V \times (W_1 \cup W_2) \setminus \{(v_1, w_1), (v_2, w_2), \ldots, (v_k, w_k)\}.$$

Suppose G has a clique $C = (V_C, E_C)$ with $|V_C| = a$. Let $U_D = V \setminus V_C$ and $V_D = E_C \cup W_1 \cup W_2$, and $D = (U_D, V_D, E_D)$ is the subgraph of G' induced by $U_D \cup V_D$. $V \setminus V_C$ is fully connected with E_C according to the definition of E'. Therefore, D is a k-defective biclique of G' with k non-edges $\{(v_1, w_1), (v_2, w_2), \ldots, (v_k, w_k)\}$, where

$$\begin{aligned} |E_D| &= |U_D \times (E_C \cup W_1)| + |(U_D \times W_2) \setminus \{(v_1, w_1) \ldots, (v_k, w_k)\}| \\ &= a(\frac{1}{2}a(a-1) + k + \frac{1}{2}a^2 - a - k) - k = a'. \end{aligned}$$

Suppose G has no clique of size a. Let $D = (U_D, V_D, E_D)$ be a k-defective biclique of G'. We then show that $|E_D|$ is always less than a'. With loss of generality we have $\{v_1, v_2, \ldots, v_k\} \subseteq U_D$ and $W_1 \cup W_2 \subseteq V_D$. This can always be achieved due to the following reasons: (1) W_2 is fully connected with U_D. (2) For any $i \in \{1, 2, \ldots, k\}$, at least one of v_i and w_i can be included in D. Moreover, if v_i (or w_i) is not in D and there is a non-edge (v, e) in D with $v \in V$ and $e \in E$, replacing v with v_i (or e with w_i) will not decrease the number of edges in D.

Let $x = |U_D|$. The number of edges in D is

$$|E_D| = |U_D| \times |V_D| - k = x \cdot (|V_D \cap E| + k + \frac{1}{2}a^2 - a - k) - k$$

where $V_D \cap E$ equals to the set of edges in $G(V \setminus U_D)$ according to our construction. There are two cases for the size of $V_D \cap E$:

(1) If $a < x \leq 2a$, let $y = x - a$. We have $|V \setminus U_D| = 2a - x = a - y$. Then $|V_D \cap E| \leq \frac{1}{2}(a-y)(a-y-1)$. Therefore, $|E_D| \leq (a + y) \cdot (\frac{1}{2}(a-y)(a-y-1) + k + \frac{1}{2}a^2 - a - k) - k$, which reduces to
$$|E_D| - a' \leq \frac{1}{2}y(y^2 + (1-a)y - 2a)$$
which is always negative with $0 < y \leq a$.

(2) If $0 \leq x \leq a$, let $y = a - x$. We have $|V \setminus U_D| = 2a - x = a + y$. As G has no clique of size a, we have $|V_D \cap E| < \frac{1}{2}(a-y)(a-y-1) - y$. Therefore, $|E_D| \leq (a-y) \cdot (\frac{1}{2}(a+y)(a+y-1) - y + k + \frac{1}{2}a^2 - a - k) - k$, which reduces to
$$|E_D| - a' < \frac{1}{2}y(-y^2 + (3-a)y)$$
which is always negative with $k \geq 1$ and $a \geq 3k$.

As a result, $|E_D|$ is always less than a' when G has no clique of size a. The proof is established.

□

2.1 Related Work

Maximum k-defective clique search. Maximum k-defective clique search is closely related to our work and has been extensively studied in recent years [8, 9, 12, 13, 20, 23, 28, 36, 46, 48]. Trukhanov et al. [48] proposed the first exact algorithm based on the Russian Doll search scheme, and Gschwinda et al. [23] further improved it with new preprocessing and verification techniques. Later studies mainly focused on branch-and-bound algorithms with stronger pruning and tighter time complexities. Chen et al. [12] proposed a branch-and-bound algorithm with a time complexity below $O^*(2^n)$ and introduced a coloring-based upper bound. Gao et al. [20] developed new vertex and edge reduction rules to improve practical efficiency. Chang [8, 9] designed non-fully-adjacent-first branching and ordering techniques, leading to tighter theoretical bounds and better empirical performance. More recently, Dai et al. [13] proposed a pivot-based branch-and-bound algorithm with several optimization techniques, achieving the state-of-the-art time complexity and experimental performance. However, these algorithms are designed to maximize the number of vertices in a k-defective clique, while the MDB problem aims to maximize the number of

edges. This divergence in optimization objectives prevents their direct application to our MDB search problem.

Maximum biclique search. Maximum biclique search has also been widely investigated [4, 10, 16, 18, 22, 37, 39, 41, 44, 47, 57], and existing studies mainly fall into three categories. The first is maximum edge biclique search, which aims to find a biclique with the maximum number of edges. Peeters et al. [41] proved its NP-hardness, and Ambühl et al. [4] studied its inapproximability. Shaham et al. [44] proposed a probabilistic algorithm based on Monte-Carlo subspace clustering, while Shahinpour et al. [45] developed an integer programming approach with scale reduction. Lyu et al. [37] proposed a branch-and-bound algorithm with progressive bounding and graph reduction techniques, enabling maximum edge biclique search on billion-scale graphs. The second category is maximum vertex biclique search, which maximizes the number of vertices and can be solved in polynomial time via a minimum-cut algorithm [21]. The third category is maximum balanced biclique search, which requires the two sides of the biclique to have equal size. Representative methods include evolutionary algorithms [57], local search frameworks [52], upper-bound propagation techniques [59], and branch-and-bound algorithms based on bipartite sparsity measurements [10]. Nevertheless, these techniques are tailored to strict biclique models, where no missing edge is allowed, and thus cannot directly handle the relaxed structure of k-MDB.

Branch-and-bound and pivoting techniques. It is worth noting that the branch-and-bound and pivoting techniques has been widely applied on many cohesive subgraph models such as the k-defective clique [8, 9, 13], the k-biplex [55, 56] and the biclique [1, 2, 11]. The state-of-the-art algorithm for maximum k-defective clique search [13] adopts a branch-and-bound framework with a pivoting strategy that selects a pivot vertex fully adjacent to the k-defective clique. However, due to the structural differences between the maximum k-defective clique and the k-MDB, no vertex can be fully adjacent to a k-defective biclique, since vertices on the same side are always disconnected. Making all the vertices on the same side adjacent would introduce a quadratic number of additional edges, leading to significant computational inefficiency. The state-of-the-art maximum k-biplex search algorithm [55] employs a pivoting strategy that selects a pivot vertex with more than k non-neighbors in the subgraph and prioritizes adding this pivot vertex with its k non-neighbors to the k-biplex, which generates at most $k+1$ sub-branches. However, there exists cases in which no vertex has more than k non-neighbors in a k-defective biclique. In such cases, directly applying this strategy to our MDB search problem may produce more than $k+1$ sub-branches, thereby incurring substantial redundant computation. The state-of-the-art maximal biclique algorithm [1] utilize a pivoting strategy that considers the number of common neighbors and selects pivot vertex in a heuristic topological order from only one side, which may not be always optimal. Although these techniques can address our MDB search problem through suitable reconstruction, the modified algorithms are still inefficient in practice.

3 Our Proposed Algorithms

In this section, we propose two novel algorithms to solve the k-MDB search problem. Specifically, we first propose a new branch-and-bound algorithm that utilizes a newly-designed binary branching strategy, achieving the worst-case time complexity of $O(m\alpha_k^n)$, where $\alpha_k < 2$ (e.g. $k = 1, 2$, and 3, we have $\alpha_k = 1.911, 1.979$, and 1.995, respectively). To further improve the efficiency, we propose a novel pivoting-based algorithm that incorporates a well-designed pivoting-based branching strategy with our proposed binary branching strategy. We prove that the time complexity of the improved algorithm is bounded by $O(m\beta_k^n)$, where $\beta_k < \alpha_k$ (e.g., when $k = 1, 2, 3$, $\beta_k = 1.717, 1.856$, and 1.931, respectively). To the best of our knowledge, the algorithms we proposed in this section are the first two k-MDB search algorithms that surpass the trivial time complexity of $O^*(2^n)$.

3.1 A New Branch-and-bound Algorithm

We firstly introduce a branch-and-bound algorithm for the k-MDB problem. It employs a straightforward branch-and-bound approach by selecting a vertex v from the graph and dividing the k-MDB search problem into two subproblems: one seeks a k-MDB that includes v, while the other seeks a k-MDB that excludes v. Specifically, given a bipartite graph $G = (U, V, E)$, let $S = (U_S, V_S)$ denote a partial set, where $G(S)$ is a k-defective biclique of G. Let $C = (U_C, V_C)$ denote a candidate set, where for any vertex $u \in C$, $G(S \cup \{u\})$ forms a larger k-defective biclique of G. Let (S, C) denote an instance of the k-MDB problem, where the goal is to find a k-MDB in $G(S \cup C)$ containing S. In particular, by setting $S = (\emptyset, \emptyset)$ and $C = (U, V)$, the initial instance $((\emptyset, \emptyset), (U, V))$ represents the global search for a k-MDB of G. When C becomes empty, the instance $(S^*, (\emptyset, \emptyset))$ yields a possible solution $G(S^*)$.

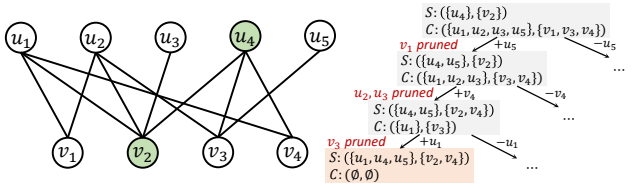
Given an instance (S, C), our branch-and-bound algorithm selects a vertex $u \in C$ and generates two new sub-instances: $(S \cup \{u\}, C \setminus \{u\})$ and $(S, C \setminus \{u\})$. To clarify, we refer to such a vertex u as a *branching vertex*. The algorithm then recursively processes each sub-instance in the same manner until C becomes empty, where S yields a possible solution. Finally, the k-MDB of G is obtained as the yielded solution with most edges. The simplest approach to obtain a branching vertex is to randomly select one from C. However, this strategy may attempt to add all the vertices in C to S, leading to many unnecessary computations. To improve the efficiency, we propose a new binary branching strategy.

The key question to improve the efficiency is how to choose a branching vertex so that the size of C can be reduced as early as possible. Intuitively, suppose there are already k non-edges in S. According to the definition of k-defective biclique, any vertex in C that has at least one non-neighbor in S can be directly pruned. Furthermore, if $v \in C$ has no non-neighbor in S, then adding v to S causes all of v's non-neighbors in C to be pruned. Consequently, we prioritize selecting a branching vertex that can introduce more non-edges to S, so that the number of non-neighbors in S reaches k as early as possible. Based on these observations, we give our new binary branching strategy as follows.

New binary branching strategy. Given an instance (S, C), let $u = \arg\max_{u \in C} \overline{d}_S(u)$. The sub-instances are generated as follows:

- If $\overline{d}_S(u) > 0$, select u as the branching vertex and generate two sub-instances: $(S \cup \{u\}, C \setminus \{u\})$, $(S, C \setminus \{u\})$.
- Otherwise, select $v \in C$ with the maximum $\overline{d}_C(v)$ as the branching vertex, and generate two sub-instances: $(S \cup \{v\}, C \setminus \{v\})$, $(S, C \setminus \{v\})$.

Example 3. Fig. 2 shows an example of our new binary branching strategy for finding a 2-MDB. The input graph is depicted in Fig. 2(a). Let the current instance be $S = (\{u_4\}, \{v_2\})$ (green vertices) and $C = (\{u_1, u_2, u_3, u_5\}, \{v_1, v_3, v_4\})$ (uncolored vertices). Fig. 2(b) illustrates



(a) the input graph (b) the branching tree

Figure 2: An example of the new binary branching strategy, where $k = 2$ and current instance is $S = (\{u_4\}, \{v_2\})$ (green vertices) and $C = (\{u_1, u_2, u_3, u_5\}, \{v_1, v_3, v_4\})$ (uncolored vertices).

the search tree guided by our strategy. Specifically, in the top-level branch, both u_5 and v_1 have exactly one non-neighbor in S, and we select u_5 as the branching vertex according to lexicographical order. Adding u_5 to S results in $|\overline{E}_{S\cup\{u_5\}}| = 1$, and v_1 is pruned from C because it now has 2 non-neighbors in $S \cup \{u_5\}$ and cannot be added to S. In the second-level branch, v_4 is selected as the branching vertex, as it is the only vertex in C with one non-neighbor in $S\cup\{u_5\}$. Adding v_4 to S introduces another non-edge to S, reaching the limit of $k = 2$. Therefore, u_2 and u_3 can be pruned from C as they still have non-neighbors in S. In the third-level branch, all vertices of C are fully adjacent to S. According to the alternative case of our binary branching rule, both u_1 and v_3 share the maximum $\overline{d}_C(\cdot)$, and we select u_1 according to lexicographical order. Finally, adding u_1 to S forces the pruning of v_3 from C, which efficiently terminates this branch.

Implementation details. Algorithm 1 outlines the pseudocode of our proposed branch-and-bound algorithm. It takes a bipartite graph G and two integers k and θ as input parameters, and returns a k-MDB D^* of G. It first initializes D^* as an empty graph, and then starts to search k-MDB by invoking the procedure `BranchB` where all vertices in G serve as the candidate set (line 2). `BranchB` holds an instance (S, C) as its parameter, where S denotes the partial set and C denotes the candidate set. Within `BranchB`, the algorithm first determines whether a larger k-defective biclique has been found (lines 5-8). After that, it selects a branching vertex u from C using the new binary branching strategy (lines 9-10) and generates two sub-instance $(S', C' \cup \{u\})$ and $(S, C \setminus \{u\})$ (lines 11-12). Here, S' and C' are the new partial set and new candidate set derived by invoking `Update` (line 11), ensuring the candidate set keeps all valid branching vertex (lines 14-15). We next analyze the time complexity of Algorithm 1.

Algorithm 1: The branch-and-bound algorithm

Input: Bipartite graph $G = (U, V, E)$, integers k and θ
Output: A k-MDB of G

```
1  D* ← an empty graph;
2  BranchB((∅, ∅), (U, V));
3  return D*;
4  Procedure BranchB(S = (U_S, V_S), C = (U_C, V_C)):
5      if C = ∅ then
6          if |E_{G(S)}| > |E_{D*}| and |U_S| ≥ θ and |V_S| ≥ θ then
7              D* ← G(S);
8          return;
9      u ← arg max_{v∈C} d̄_S(v);
10     if d̄_S(u) = 0 then u ← arg max_{v∈C} d̄_C(v) ;
11     (S', C') ← Update(S, C, u);
12     BranchB(S' ∪ {u}, C'); BranchB(S, C \ {u});
13 Function Update(S, C, u):
14     C' ← {v ∈ C \ {u} | d̄_{S∪{u}}(v) ≤ k − d̄(S)};
15     C'_0 ← {v ∈ C' | d̄_{S∪{u}∪C'}(v) = 0};
16     return (S ∪ C'_0, C' \ C'_0);
```

Theorem 4. Given a bipartite graph $G = (U, V, E)$ and an integer k, the time complexity Algorithm 1 is given by $O(m\alpha_k^n)$, where α_k is the largest real root of equation $x^{2k+5} - 2x^{2k+4} + x^3 - x^2 + 1 = 0$. Specifically, when $k = 1, 2$ and 3, we have $\alpha_k = 1.911, 1.979$ and 1.995, respectively.

Proof. Let $T(n, l)$ denote the maximum number of leaf instances generated by `BranchB` (i.e., instances with an empty candidate set C), where $n = |C|$ represents the number of candidate vertices and $l = k - |\overline{E}(S)|$ represents the remaining number of non-edges can be added to S. As $l \le k$, we have that $T(n, l) \le T(n, k)$. Since each recursive call of `BranchB` takes $O(m)$ time, the total time complexity of `BranchB` is $O(m \cdot T(n, k))$.

We next analyze $T(n, k)$ based on the binary branching strategy. There are two cases in each branch of `BranchB`:

(1) **Case 1:** $\max_{v\in C} \overline{d}_S(v) \ge 1$. We select $u = \arg\max_{v\in C} \overline{d}_S(v)$. Adding u to S introduces at least one non-edge to S, so the corresponding recurrence is $T(n, k) \le T(n-1, k) + T(n-1, k-1) \le \sum_{i=1}^{k} T(n-i, k-i+1) + T(n-k, 0)$. As any $v \in C$ satisfying $\overline{d}_S(v) \ge 1$ can be pruned when $l = 0$, the term $T(n-k, 0)$ is eliminated. Therefore, the recurrence for this case is given by

$$T(n, k) \le \sum_{i=1}^{k} T(n-i, k-i+1). \tag{1}$$

(2) **Case 2:** $\max_{v\in C} \overline{d}_S(v) = 0$. We select $u = \arg\max_{v\in C} \overline{d}_C(v)$, and the corresponding recurrence is $T(n, k) \le T(n-1, k) + T(n-1, k)$. Note that $\overline{d}_C(u) > 0$ (otherwise, $S\cup C$ already forms a valid k-MDB), which ensures that the sub-branch adding u to S triggers Case 1. Consequently, the recurrence for this case is given by $T(n, k) \le T(n-1, k) + T(n-2, k) + T(n-2, k-1) \le \sum_{i=1}^{2k} T(n-i, k-\lfloor \frac{i-1}{2} \rfloor) + T(n-2k, 0)$. Considering that when $l = 0$, adding any $v \in C$ with $\overline{d}_C(u)$ to S prunes at least one vertex of C, so we have $T(n-2k, 0) \le T(n-2k-1, 0) + T(n-2k-2, 0) \le T(n-2k-1, 0) + T(n-2k-3, 0) + T(n-2k-4, 0)$. Therefore, the recurrence for this case is given by

$$T(n, k) \le \sum_{i=1}^{2k+1} T(n-i, k-\lfloor \tfrac{i-1}{2} \rfloor) + T(n-2k-3, 0) + T(n-2k-4, 0). \tag{2}$$

The overall worst-case complexity is governed by recurrence (2). By utilizing the algebraic technique from [19], $T(n, k)$ can be expressed in the form of $O(\alpha_k^n)$, where the value of α_k is the largest real root of the equation $x^n = x^{n-1} + x^{n-2} + \cdots + x^{n-2k-1} + x^{n-2k-3} + x^{n-2k-4}$, which simplifies to $x^{2k+5} - 2x^{2k+4} + x^3 - x^2 + 1 = 0$. This completes the proof. □

Lemma 2. Given a bipartite graph $G = (U, V, E)$, the time complexity of Algorithm 1 for finding a 0-MDB of G is $O(m \times 1.618^n)$.

Proof. When $k = 0$, the vertex $v \in C$ with maximum $\overline{d}_C(v)$ is selected as the branching vertex. Moving v from C to S results in the deletion of v's non-neighbors in C, leading to the recurrence $T(n) \le T(n-1) + T(n-2)$, which yields $T(n) = 1.618^n$. □

3.2 A Novel Pivoting-based Algorithm

We observe that Algorithm 1 may still generate numerous unnecessary sub-instances. Consider an instance (S, C) and a branching vertex $v \in C$ with $\overline{d}_S(v) = 0$. In Algorithm 1, if v and all non-neighbors of v have been excluded from the current instance, the remaining vertices cannot form any k-MDB. This is because any k-defective biclique derived from these remaining vertices can always be expanded by including v. Consecutively, Algorithm 1 may produce many k-defective bicliques that are not maximum. To address the problem and further improve the efficiency, we propose a novel pivoting-based algorithm in this section. We formalize above insights into the following lemma.

Lemma 3. Given an instance (S, C) and any vertex $v \in C$ with $\overline{d}_S(v) = 0$, the k-MDB of instance (S, C) must contain either v or a vertex in $\overline{N}_C(v)$.

Based on Lemma 3, we develop a pivoting strategy that provides a new way to generate sub-instances. The details are presented as follows.

Pivoting strategy. Consider an instance (S, C) and a vertex $u \in C$ with $\overline{d}_S(u) = 0$. Let u be the *pivot vertex*. Assume $\overline{N}_C(u) = \{v_1, v_2, \ldots, v_r\}$, where $r = \overline{d}_C(u)$. The pivoting strategy treats each vertex in $\{u\} \cup \overline{N}_C(u)$ as a branching vertex and generates exactly $r + 1$ sub-instances. The first sub-instance is given by $(S \cup \{u\}, C \setminus \{u\})$, and for $i \geq 2$, the i-th sub-instance is given by $(S \cup \{v_{i-1}\}, C \setminus \{u, v_1, v_2, \ldots, v_{i-1}\})$.

With this strategy, S is expanded only with the vertex u and vertices in $\overline{N}_C(u)$, avoiding redundant sub-instances that would lead to non-maximum k-defective biclique. Next, we consider the selection of the pivot vertex. Let $C_0 \subseteq C$ be the set of vertices fully connected to S, i.e., $C_0 = \{v \in C \mid \overline{d}_S(v) = 0\}$. According to the pivoting strategy, any vertex from C_0 can serve as a pivot vertex. To minimize the number of sub-instances, the vertex $u \in C_0$ with the fewest non-neighbors in C is chosen as the pivot vertex.

In the case where C_0 is empty, every vertex in C has at least one non-neighbor in S, making the pivoting strategy inapplicable. Fortunately, as discussed in Sec. 3.1, continuously branching k times by adding vertices from C to S will result in S containing at least k non-edges. Moreover, C becomes an empty set after this process, which significantly reduces unnecessary computations. Motivated by this observation, we integrate the pivoting strategy with the binary branching strategy to develop a novel pivoting-based branching strategy as outlined below.

Novel pivoting-based branching strategy. Given an instance (S, C), let $C_0 = \{v \in C \mid \overline{d}_S(v) = 0\}$. The new sub-instances of (S, C) are generated in the following way:

- If C_0 is empty or $|C \setminus C_0| > k - |\overline{E}_S|$, let $u = \arg\max_{v \in C} \overline{d}_S(v)$ be the branching vertex and generate two sub-instances: $(S \cup \{u\}, C \setminus \{u\})$ and $(S, C \setminus \{u\})$.
- Otherwise, let $u = \arg\min_{v \in C_0} \overline{d}_C(v)$, and then:
  - If $\overline{d}_C(u) > k - |\overline{E}_S| > 0$, generate two sub-instances with u as the branching vertex: $(S \cup \{u\}, C \setminus \{u\})$ and $(S, C \setminus \{u\})$.
  - Otherwise, assume $\overline{N}_C(u) = \{v_1, v_2, \ldots, v_r\}$, where $r = \overline{d}_C(v)$, and generate $\overline{d}_C(u) + 1$ sub-instances with u as the pivot vertex, where the first sub-instance is given by $(S \cup \{u\}, C \setminus \{u\})$, and for $i \geq 2$, the i-th sub-instance is given by $(S \cup \{v_{i-1}\}, C \setminus \{u, v_1, v_2, \ldots, v_{i-1}\})$.

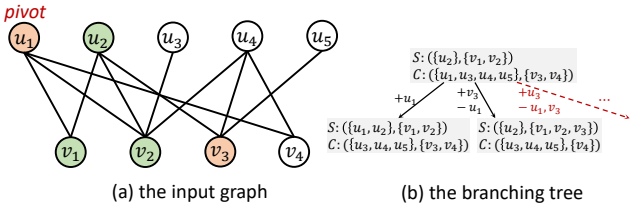


Figure 3: An example of the pivoting-based branching strategy, where $k = 2$ and current instance is set as $S = (\{u_2\}, \{v_2\})$ (green vertices) and $C = (\{u_1, u_3, u_4, u_5\}, \{v_1, v_3, v_4\})$ (orange and uncolored vertices).

Example 5. Fig. 3 shows an example of the our pivoting-based branching strategy for the 2-MDB search. The example graph is given by Fig. 3(a), with the current instance set as $S = (\{u_2\}, \{v_1, v_2\})$ (green vertices) and $C = (\{u_1, u_3, u_4, u_5\}, \{v_3, v_4\})$ (orange and uncolored vertices). According to the pivoting-based branching strategy, $C_0 = (\{u_1\}, \{v_3\})$, and u_1 is selected as the pivot vertex since it has the fewest non-neighbors in C among vertices in C_0. Then, only u_1 and all non-neighbors of u_1 in C are used to generate new sub-instances (orange vertices), while other vertices of C are not considered (uncolored vertices). This results in only two sub-instances generated by our pivot-based branching strategy. The branching tree for this example is illustrated in Fig. 3(b), where the black arrows represent the generated branches while the red arrows indicate the unnecessary branches.

Implementation details. Algorithm 2 outlines the pseudocode of the proposed pivot-based algorithm. It begins by searching for k-MDB by invoking the procedure `BranchP` with an empty partial set S and the entire vertex set of G as the candidate set C (line 2). The algorithm returns the global variable D^* as the final k-MDB after `BranchP` is processed (line 3). Within `BranchP`, the algorithm first updates D^* if a larger k-defective biclique is found (lines 5-7). Then, it generates new sub-instances based on the pivoting-based branching strategy (lines 8-22). Specifically, if $C_0 = \emptyset$ or $|C \setminus C_0| > k - |\overline{E}_S|$, the vertex $u = \arg\max_{v \in C} \overline{d}_S(v)$ is selected as the branching vertex (line 11), and two new sub-instances are generated (line 13). Otherwise, the algorithm assigns $u = \arg\min_{v \in C_0} \overline{d}_C(v)$. If $\overline{d}_C(u) > k - |\overline{E}_S|$, two sub-instances are generated with u as the branching vertex (lines 16-18). Otherwise, $\overline{d}_C(u) + 1$ new sub-instances are generated with u and all u's non-neighbors in C as branching vertex (lines 20-22). When a branching vertex is added to S, the algorithm adopts the `Update` procedure presented in Algorithm 1 to update the current instance (lines 12, 17 and 21). Below, we analyze the time complexity of Algorithm 2.

Theorem 6. Given a bipartite graph $G = (U, V, E)$ and an integer k, the time complexity of Algorithm 2 is given by $O(m\beta_k^n)$, where β_k is the largest real root of equation $x^{2k+5} - 2x^{2k+4} + x^{k+3} - 2x + 2 = 0$. Specifically, when $k = 1, 2$ and 3, we have $\beta_k = 1.717, 1.856$ and 1.931, respectively.

Proof. Let $T(n, l)$ denote the maximum number of leaf instances generated by `BranchP`, where $n = |C|$ and $l = k - |\overline{E}_S|$ (i.e. the remaining number of non-edges allowed in S). Since $l \leq k$, we have $T(n, l) \leq T(n, k)$. Finding the vertex with the most (or least) non-neighbors takes $O(m)$ time per recursive call, so `BranchP` takes $O(m \cdot T(n, k))$ time in total. We analyze $T(n, k)$ based on the three cases of the pivoting-based branching strategy:

Algorithm 2: The pivoting-based algorithm

```
Input: Bipartite graph G = (U, V, E), integers k and θ
Output: A k-MDB of G
1  D* ← an empty graph;
2  BranchP((∅, ∅), (U, V));
3  return D*;
4  Procedure BranchP(S = (U_S, V_S), C = (U_C, V_C)):
5      if C = ∅ then
6          if |E_{G(S)}| > |E_{D*}| and |U_S| ≥ θ and |V_S| ≥ θ then
7              D* ← G(S);
8          return;
9      C_0 ← {v ∈ C | d̄_S(v) = 0};
10     if C_0 = ∅ or |C \ C_0| > k − |Ē_S| then
11         u ← arg max_{v∈C} d̄_S(v);
12         (S', C') ← Update(S, C, u);
13         BranchP(S' ∪ {u}, C'); BranchP(S, C \ {u});
14     else
15         u ← arg min_{v∈C_0} d̄_C(v);
16         if d̄_C(u) > k − |Ē_S| > 0 then
17             (S', C') ← Update(S, C, u);
18             BranchP(S' ∪ {u}, C'); BranchP(S, C \ {u});
19         else
20             for v ∈ {u} ∪ N̄_C(u) do
21                 (S', C') ← Update(S, C, v);
22                 BranchP(S' ∪ {v}, C'); C ← C \ {v};
```

(1) **Case 1:** $C_0 = \emptyset$ **or** $|C \setminus C_0| > l$**.** BranchP continuously applies the binary branching strategy. After moving at most k vertices from C to S, $k - |\overline{E}(S)|$ reaches 0. At this point, at least one vertex is pruned from C due to having non-neighbors in S. Thus, the recurrence is bounded by:

$$T(n,k) \leq \sum_{i=1}^{k+1} T(n-i, k-i+1). \tag{3}$$

(2) **Case 2:** $l = 0$**.** The problem degenerates to maximum biclique search using the pivoting strategy. Let $r = \overline{d}_C(u)$. The strategy generates $r+1$ sub-instances, yielding $T(n,0) \leq T(n-r-1,0) + \sum_{i=1}^{r} T(n-r-i,0)$. If $r = 1$, the recurrence simplifies to $T(n,0) \leq 2T(n-2,0)$. If $r \geq 2$, following existing algebraic technique [19], solving this recurrence equivalents to solving $x^n = x^{n-r-1} + \sum_{i=1}^{r} x^{n-r-i}$, which yields a maximum real root of $\gamma_2 = 1.395 < \sqrt{2}$ (for $r \geq 2$). Therefore, the recurrence for this case can always be bounded by:

$$T(n,0) \leq 2T(n-2,0). \tag{4}$$

(3) **Case 3:** $C_0 \neq \emptyset$ **and** $0 < |C \setminus C_0| \leq l$**.** Let $u = \arg\min_{v \in C_0} \overline{d}_C(v)$. If $\overline{d}_C(u) \leq l$, BranchP uses u as the pivot vertex, generating at most $\overline{d}_C(u) + 1 \leq l + 1$ sub-instances, yielding $T(n,k) \leq \sum_{i=1}^{k+1} T(n-i,k)$. Otherwise, if $\overline{d}_C(u) > l$, BranchP uses u for binary branching. Adding u to S triggers Case 1 in the next level since $|C \setminus C_0| > l - |\overline{E}_{S\cup\{u\}}|$. The alternative branches eventually reduce to $l = 0$ (Case 2). Substituting $T(n-2k-2,0) \leq 2T(n-2k-4,0)$ from Case 2, the worst-case recurrence is bounded by:

$$T(n,k) \leq \sum_{i=1}^{k+1} T(n-i,k) + 2T(n-2k-4,0). \tag{5}$$

The overall worst-case complexity is governed by Case 3. By utilizing the algebraic technique from [19], let $T(n,k) = O(\beta_k^n)$, where β_k is the largest real root of the equation $x^n = \sum_{i=1}^{k+1} x^{n-i} + 2x^{n-2k-4}$. This simplifies to $x^{2k+5} - 2x^{2k+4} + x^{k+3} - 2x + 2 = 0$, which completes the proof. □

Lemma 4. Given a bipartite graph $G = (U, V, E)$, the time complexity of Algorithm 2 for finding a 0-MDB of G is $O(m \cdot 1.414^n)$.

Proof. When setting $k = 0$, Algorithm 2 will always execute the pivoting-based branching strategy (lines 20-22), corresponding to a recurrence of $T(n) < 2T(n-2)$ according to the proof of Theorem 6. Therefore, we have $T(n) = 1.414^n$. □

Discussion. It is worth noting that the branch-and-bound and pivoting techniques has been widely applied on many cohesive subgraph models such as the k-defective clique [8, 9, 13], the k-biplex [55, 56] and the biclique [1, 2, 11]. However, our binary branching strategy and pivoting-based branching strategy are significantly different from all these existing solutions.

Compared to the state-of-the-art maximum k-defective clique algorithm in [13], which achieves a non-trivial time complexity by utilizes a condition to restrict the scenarios of pivoting, directly applying this approach to our pivoting-based algorithm would degrade the time complexity to $O^*(2^n)$. In contrast, we employ a more refined pivoting strategy that considers both the number of non-edges between S and C (lines 10-13 in Algorithm 2) and the number of pivot vertex's non-neighbors (lines 15-18 in Algorithm 2), thereby effectively bounding the time complexity.

The state-of-the-art algorithm for maximum k-defective clique search [13] adopts a branch-and-bound framework with a pivoting strategy that selects a pivot vertex fully adjacent to the k-defective clique. However, due to the structural differences between the maximum k-defective clique and the k-MDB, no vertex can be fully adjacent to S (as vertices on the same side are always disconnected), so directly applying this strategy to our MDB search problem cannot reduce the worst-case time complexity. In contrast, we propose a pivoting technique that considers not only the number of non-edges between S and C on both sides (lines 10-13 in Algorithm 2) but also the non-neighbor relationships between the pivot vertex and vertices on the opposite side (lines 15-18 in Algorithm 2), thereby better aligning with the structural characteristics of k-MDB.

Compared to the state-of-the-art pivoting-based maximum biplex search algorithm in [55], which achieves a time complexity of $O(my_k^n)$, with $\gamma_k = 1.754$ when $k = 1$, our algorithm MDBP reaches a better time complexity. This is mainly because the algorithm in [55] only considers prioritizing k non-neighbors of S, which is similar to our binary branching strategy in Sec. 3.1. By contrast, our pivoting-based branching strategy also bounds the case that fewer than k non-neighbors are between S and C, which reduces the size of branching tree and thereby improves the time complexity (see the proof of Theorem 6). The state-of-the-art maximum k-biplex search algorithm [55] employs a pivoting strategy that selects a pivot vertex with more than k non-neighbors in the subgraph and prioritizes adding this pivot vertex with its k non-neighbors to the k-biplex. However, this approach cannot be directly applied to k-MDB search, since there exists cases in which no vertex has more than k non-neighbors in $S \cup C$. In contrast, we propose a completely different pivoting strategy, which selects a pivot vertex with at most k non-neighbors in C and then generates at most $k+1$ sub-branches. This design explicitly handles cases where fewer than k non-edges are between S and C, thereby enabling our algorithm to effectively solve the k-MDB search problem.

The state-of-the-art maximal biclique algorithm [1] utilize a pivoting strategy that considers the number of common neighbors and selects pivot vertex in a heuristic topological order from only one side, which may not be always optimal. In contrast, our pivoting-based branching strategy selects pivot vertex from both side through deterministic condition, which always generates fewest new sub-branches in each recursion and thus reduces the size of branching tree.

4 Optimization Techniques

In this section, we propose various optimization techniques, including graph reduction techniques to eliminate redundant vertices, upper-bounding techniques to prune unnecessary instances, and a heuristic approach for identifying an initial large k-defective biclique. These techniques can be easily integrated into our proposed algorithms in Sec. 3, and can significantly enhance their efficiency.

4.1 Graph Reduction Techniques

Common-neighbor-based reduction. For two vertices u and v on the same side, let $cn(u, v)$ (or $cn_S(u, v)$) denote the number of common neighbors of u and v in G (or in S). Given a bipartite graph $G = (U, V, E)$ and a k-MDB $D = (U_D, V_D, E_D)$ of G, every edge $(u, v) \in E_D$ satisfies that (1) $d_D(u) \geq \theta - k$, and (2) for all $w \in N_D(v)$, $cn_D(u, w) \geq \theta - k$. By combining the two conditions, we can conclude that for an edge $(u, v) \in E$, if v has fewer than $\theta - k$ neighbors w such that $cn(u, w) \geq \theta - k$, the edge (u, v) can be pruned in G.

Based on this idea, we present Algorithm 3 to prune such edges from G, referred to as `CnReduction`. The algorithm enumerates all edges (u, v) in ascending order of u's degree (lines 1-2). For each edges (u, v), the algorithm counts common neighbors between u and the neighbors w of v (lines 3-8). If v has fewer than $\theta - k$ neighbors w satisfying $cn(u, w) \geq \theta - k$, the edge (u, v) is pruned from G (lines 9-11). Additionally, if the degree of u or v drop below $\theta - k$ due to the deletion of (u, v), the algorithm will also prune u or v from G (lines 12-13). We give the time complexity of Algorithm 3 in the following lemma.

Lemma 5. Given a bipartite graph $G = (U, V, E)$, the complexity of Algorithm 3 is $O(d \cdot m)$, where $d = \max_{u \in U \cup V} d(u)$.

Proof. In algorithm 3, the degree ordering calculation at line 1 takes $O(n)$ time. The three loops at line 3-5, lines 6-8 and lines 9-11 run the same rounds and all have a time complexity of $O(d \cdot m)$. The deletion of vertices at line 12 and line 13 require maintaining the degree of their neighbors, which takes $O(d)$ time, so the loop at line 13 totally takes at worst $O(d \cdot m)$ time. □

In practice, Algorithm 3 performs efficiently although its time complexity is $O(d \cdot m)$. This is because in real-world graphs, most vertices have degrees much smaller than d. As a result, the actual runtime of the algorithm often does not reach this worst-case time complexity.

One-non-neighbor reduction. For an instance (S, C) and a vertex u, let C_u denote the subset of C where all vertices are on the same side as u. If a branching vertex has only one non-neighbor in $S \cup C$, removing it from C can lead to the pruning of additional vertices. The following lemma explains this idea in detail.

Theorem 7. Given an instance (S, C) and a vertex $u \in C$ with $\overline{d}_{S \cup C}(u) = 1$, if u is removed from C, all vertices $v \in C_u$ with $\overline{d}_S(v) \geq 1$ can be pruned from C. Moreover, if $d_C(u) = 1$ and w is the unique non-neighbor of u, then all vertices in $\overline{N}_C(w)$ can also be pruned from C.

Algorithm 3: `CnReduction`(G, k, θ)

Input: Bipartite graph $G = (U, V, E)$, integers k and θ
Output: A reduced bipartite graph

```
1  O ← the ascending degree order of U ∪ V;
2  for u ∈ O do
3      for v ∈ N(u) do
4          for w ∈ N(v) do
5              cn(w) ← 0;
6      for v ∈ N(u) do
7          for w ∈ N(v) do
8              cn(w) ← cn(w) + 1;
9      for v ∈ N(u) do
10         N' ← {w ∈ N(v) | cn(w) ≥ θ − k};
11         if |N'| < θ − k then E ← E \ (u, v) ;
12     if d(u) < θ − k then U ← U \ u;
13     for v ∈ N(u) s.t. d(v) < θ − k do V ← V \ v ;
14 return G;
```

Proof. Assume D is a k-MDB derived from (S, C) excluding u. For any $v \in C_u$ such that $v \in D$ and $\overline{d}_S(v) \geq 1$, we have $D \setminus \{v\} \cup \{u\}$ is also a k-MDB. In the case where $\overline{d}_C(u) = 1$ and w is the unique non-neighbor of u, if $w \notin D$, then $D \cup \{u\}$ forms a larger k-MDB. Otherwise, for any $v \in \overline{N}_C(w)$ such that $v \in D$, we have $D \setminus \{v\} \cup \{u\}$ is also a k-MDB. □

Ordering-based reduction. Given a bipartite graph $G = (U, V, E)$, let $u_1, u_2, \ldots, u_{|U|}$ be an arbitrary vertex order of U. For each vertex u_i, the ordering-based reduction aims to find a k-MDB D_i including u_i while excluding $u_1, u_2, \ldots, u_{i-1}$. Obviously, the k-MDB of G is the one with most edges among $D_1, D_2, \ldots, D_{|U|}$. Next, we extend Lemma 1 to constrain the vertex distance in D_i, as shown in the following lemma.

Lemma 6. Given a k-defective biclique $D = (U_D, V_D, E_D)$, if $|U_D| > k$ and $|V_D| > k$, the maximum distance between any two vertices of D is 3.

Lemma 6 indicates that when searching for D_i, vertices with distances more than 3 from u_i can be pruned. Let $N^2(u) = \{w \in N(v) \mid v \in N(u)\}$, i.e., the set of u's 2-hop neighbors. Let $N^2_+(u_i) = N^2(u_i) \cap \{u_{i+1}, u_{i+2}, u_{|U|}\}$. According to Lemma 6, we can construct an initial instance (S_i, C_i) to find D_i, where $S_i = (\{u_i\}, \emptyset)$ and $C_i = (N^2_+(u_i), \bigcup_{u_j \in N^2_+(u_i)} N(u_j))$.

In practical implementation, to identify a larger k-defective biclique as early as possible, we traverse the vertices in U in descending order of their degrees. This approach is based on the intuition that vertices with higher degrees are more likely to be present in an MDB. Additionally, the common-neighbor-based reduction techniques can be employed on (S_i, C_i) to prune unnecessary vertices and edges. Specifically, vertices $u \in C_i$ are pruned if $d_{S_i \cup C_i}(u) < \theta - k$, and edges (u, v) are pruned if there are fewer than $\theta - k$ neighbors w of v satisfying $cn(u, w) \geq \theta - k$.

Progressive bounding reduction. The size threshold θ is provided as an input parameter and can be set to a very small value (e.g., $= k+1$), which may limit the effectiveness of our optimization techniques. To address this issue, we utilize the progressive bounding technique inspired from the maximum biclique problem [37] to provide tighter size thresholds for the k-MDB search problem.

Specifically, let θ^t_U and θ^t_V denote the size thresholds in t-th round. The objective of t-th round is to identify a k-MDB D_t =

$(U_{D_t}, V_{D_t}, E_{D_t})$ satisfying $|U_{D_t}| \geq \theta_U^t$ and $|V_{D_t}| \geq \theta_V^t$. Let $D^* = \max\{D_1, D_2, \ldots, D_{t-1}\}$ be the currently largest k-defective biclique. The progressive bounding reduction generates θ_U^t and θ_V^t in the following way:

$$\theta_U^t = \max\{\theta, \lfloor \theta_U^{t-1}/2 \rfloor\}, \tag{6}$$

$$\theta_V^t = \max\{\theta, \lfloor |E_{D^*}|/\theta_U^{t-1} \rfloor\}. \tag{7}$$

Specially, $\theta_U^0 = \max_{u \in U} d_U(u) + k$, and the iteration terminates when $\theta_U^t = 0$. The following lemma shows the correctness of the progressive bound reduction.

Lemma 7. Given a bipartite graph G and a k-MDB $D = (U_D, V_D, E_D)$ of G, there always exists an integer t such that $\theta_U^t \leq |U_D|$ and $\theta_V^t \leq |V_D|$.

In many real-world graphs, θ_U^t and θ_V^t are often significantly larger than θ, which helps facilitate more effective graph reduction, such as a smaller reduced graph by performing the common-neighbor-based reduction.

Discussion. Several prior studies have proposed similar graph reduction techniques for other subgraph models [13, 14, 49, 55, 58]. The progressive bounding reduction have been widely applied on these models [49, 55, 58]. In this work, we adapted these techniques to our problem by incorporating the unique properties of k-defective bicliques. However, our proposed common-neighbor-based reduction and one-non-neighbor reduction are technically distinct from all prior approaches.

On the one hand, the common-neighbor-based reduction leverages the constraints on vertex degree and common neighbors in a k-MDB. The reduction based on vertex degree has been widely applied in the form of core [55, 58]. However, this approach results in a very limited reduction in graph size. Additionally, by applying a single constraint on common neighbors of two vertices u and v where $cn(u, v) < \theta - k$, we can conclude that u and v cannot coexist in the same k-MDB. However, maintaining such coexistence relationships requires $O(n^2)$ space, making it quite challenging for practical implementation.

On the other hand, the one-non-neighbor reduction focuses on analyzing vertices in C that have only one non-neighbor in the current subgraph $G(S \cup C)$. A relevant prior technique addresses the maximum k-defective clique search problem [13], where a vertex in C with exactly one non-neighbor in the subgraph can be directly added to S. However, this reduction cannot be directly applied to the k-MDB problem, as such a vertex may not belong to any k-MDB. For example, consider the following 1-MDB search instance (S, C), where $|U_S| = 2$, $|V_S| = 3$, $U_C = \{u\}$ and $V_C = \{v\}$. $G(S)$ has one non-edge, u and v are non-neighbors. Directly adding v to S results in a non-maximum solution, because $G(S \cup \{v\})$ has 7 edges while adding u to S yields a k-defective biclique with 8 edges. In contrast to [13], we focus on the deletion of such vertices rather than their addition, which restricts the pruning scope to vertices on the same side (as discussed in Theorem 7), thereby ensuring the correctness of the searching result.

4.2 Novel Upper-Bounding Techniques

In this subsection, we propose a novel upper-bounding technique that provides effective upper bounds for both vertices and edges. Formally, given an instance (S, C), assume $u_1, u_2, \cdots, u_{|U_C|}$ and $v_1, v_2, \cdots, v_{|V_C|}$ are the vertices of U_C and V_C in ascending order of $\overline{d}_S(\cdot)$, respectively. Let $\overline{d}_S^i(U_C) = \sum_{t=1}^{i} \overline{d}_S(u_i)$ and $\overline{d}_S^i(V_C) = \sum_{t=1}^{i} \overline{d}_S(v_i)$, i.e., the total number of non-neighbors in S among the first i vertices of U_C and V_C. Based on this, the vertex upper bound is presented in the following lemma.

Algorithm 4: `UpperBound`$((S, C), k)$

Input: Instance $(S = (U_S, V_S), C = (U_C, V_C))$ and integer k
Output: two vertex upper bounds and an edge upper bound

1 $k' \leftarrow k - |\overline{E}_S|$;
2 Let $u_1, u_2, \cdots, u_{|U_C|}$ and $v_1, v_2, \cdots, v_{|V_C|}$ be the vertices of U_C and V_C in ascending order of $\overline{d}_S(\cdot)$;
3 $\overline{d}_U \leftarrow 0$; $\overline{d}_V \leftarrow 0$; $c_U \leftarrow 0$; $c_V \leftarrow 0$;
4 **for** $i \leftarrow 1 \ldots |U_C|$ *s.t.* $\overline{d}_U + \overline{d}_S(u_i) \leq k'$ **do**
5 $\overline{d}_U \leftarrow \overline{d}_U + \overline{d}_S(u_i)$;
6 $c_U \leftarrow c_U + 1$;
7 $e \leftarrow (|U_S| + c_U) \cdot |V_S| - |\overline{E}_S| - \overline{d}_U$;
8 $i \leftarrow c_U$;
9 **for** $j \leftarrow 1 \ldots |V_C|$ *s.t.* $\overline{d}_V + \overline{d}_S(v_i) \leq k'$ **do**
10 **while** $\overline{d}_U + \overline{d}_V + j > k'$ **do**
11 $\overline{d}_U \leftarrow \overline{d}_U - \overline{d}_S(u_i)$;
12 $i \leftarrow i - 1$;
13 $\overline{d}_V \leftarrow \overline{d}_V + \overline{d}_S(u_i)$;
14 $c_V \leftarrow c_V + 1$;
15 $e \leftarrow \max\{e, (|U_S| + i) \cdot (|U_S| + j) - |E_S| - \overline{d}_U - \overline{d}_V\}$;
16 **return** $|U_S| + c_U, |V_S| + c_V, e$;

Lemma 8 (Vertex upper bounds). For any k-MDB $D = (U_D, V_D, E_D)$ derived from (S, C), we have $|U_D| \leq |U_S| + i$ and $|V_D| \leq |V_S| + j$, where i and j are the largest values satisfying $\overline{d}_S^i(U_C) \leq k - |\overline{E}_S|$ and $\overline{d}_S^j(V_C) \leq k - |\overline{E}_S|$, respectively.

Proof. For the side of U, it is obvious that i is the maximum number of vertices that can be added from U_C to U_S such that $(U_S \cup \{u_1, u_2, \ldots, u_i\}, V_S)$ remains a valid k-defective biclique. For any k-MDB derived from (S, C) consisting of $(U_S \cup U_{C'}, V_S \cup V_{C'})$, $(U_S \cup U_{C'}, V_S)$ is still a k-defective biclique. Therefore, we have that $|U_{C'}| \leq i$. The proof on the other side is similar. □

Based on Lemma 8, if both sides reach their vertex upper bounds, an edge upper bound can be calculated as $(|U_S| + i) \times (|V_S| + j) - |\overline{E}_S| - \overline{d}_S^i(U_C) - \overline{d}_S^j(V_C)$. However, this upper bound does not account for the coexistence of vertices in U_C and V_C. To solve the issue, we present a tighter edge upper bound in the following lemma.

Lemma 9 (Edge upper bound). For any k-MDB D derived from (S, C), we have that $|E_D| \leq \max_{0 \leq i \leq |U_C|}(|U_S| + i) \times (|V_S| + j) - |\overline{E}_S| - \overline{d}_S^i(U_C) - \overline{d}_S^j(V_C)$, where j is the largest value satisfying $\overline{d}_S^i(U_C) + \overline{d}_S^j(V_C) \leq k - |\overline{E}_S|$.

Proof. Consider a new instance (S, C') reconstructed from (S, C) by fully connecting vertices between $U_{C'}$ and $V_{C'}$, i.e., $\overline{E}_C = \emptyset$. The ordering of vertices in C' with respected to $\overline{d}_S(\cdot)$ remains unchanged. There exists a k-MDB derived from (S, C') composed of S, a prefix of $\{u_1, u_2, \ldots, u_{|U_C|}\}$, and a prefix of $\{v_1, v_2, \ldots, v_{|V_C|}\}$ (otherwise, for any vertex not in a prefix, a preceding vertex can always be found to replace it). Moreover, the k-MDB derived from (S, C') have more edges than any k-MDB derived from (S, C). Consequently, Lemma 9 provide a correct edge upper bound. □

Implementation details. Algorithm 4 presents a pseudocode of our upper-bounding techniques. It starts by setting k' as the remaining number of missing edges (line 1), and sorting the vertices of U_C and V_C in ascending order of $\overline{d}_S(\cdot)$ (line 2), which can be efficiently done using a bucket sort in linear time and space. Then,

the algorithm calculates the vertex upper bound in U by finding the longest prefix of U_C that can be added to U_S (lines 4-6), where c_U records the prefix length. After that, the algorithm derives the upper bound c_V for vertices in V using the same way, and simultaneously calculates the upper bound e for edges by removing vertices from the end of the prefix of U_C while computing the longest prefix of V_C (lines 9-15). Finally, it returns two upper bounds of both sides as well as an edge upper bound. We present the time complexity of the algorithm in the following lemma.

Lemma 10. The time complexity of Algorithm 4 is $O(n)$.

Proof. As discussed, line 2 of Algorithm 4 can be implemented with a bucket sort, which takes $O(n)$ time. Lines 4-6 clearly takes $O(n)$ time. Also, it is easy to show that both two loops at lines 10-12 and lines 13-15 consume at most $O(n)$ time. Therefore, Algorithm 4 has a time complexity of $O(n)$. □

4.3 A Heuristic Algorithm

We propose a heuristic algorithm to efficiently calculate a large k-defective biclique. We observe that in many real-world graphs, one side of the k-MDB with size threshold θ contains exactly θ vertices, while the other side contains much more than θ vertices. Motivated by this, our heuristic algorithm aims to find a large k-defective biclique with one side has θ vertices, which is presented in Algorithm 5. The heuristic algorithm attempts to calculate two k-defective bicliques given by (U_0, V_0) and (U_1, V_1), where $|U_0| = \theta$ and $|V_1| = \theta$ (lines 1-2). The one with more edges will be returned as the large k-defective biclique (lines 3-4). The Expand procedure aims to compute a k-defective biclique with θ vertices on the side of U. Specifically, it initializes two vertex sets $U' =$ and $V' = V$ (line 6). The algorithm then iteratively attempts to add θ vertices to U' (lines 7-18). In each iteration, the vertex $u \in U$ with the highest degree in V' is selected for addition (line 8). If $d_{V'}(u) < \theta - k$, the iteration terminates early, as no more vertex in U can be added to U' (line 9). Then, the algorithm determines whether u can be added to U' by ensuring that there are θ vertices in V' that can form a k-defective biclique with $U' \cup \{u\}$ (lines 10-13). If the condition is satisfied, the algorithm repeatedly removes vertices from V' that have the most non-neighbors in $U' \cup \{u\}$, until u can be added to U' (lines 14-18). Finally, if U' contains fewer than θ vertices, the algorithm returns an empty result. Otherwise, the algorithm yields (U', V') as a large k-defective biclique with $|U'| = \theta$. Note that finding a vertex with maximum (minimum) degree (lines 8, 12 and 16) can be implemented in constant time using the decomposition techniques similar to [5]. We present the time complexity of Algorithm 5 in the following lemma.

Lemma 11. Given a bipartite graph $G = (U, V, E)$ and two integers k, θ, the time complexity of Algorithm 5 is $O(\theta n + m)$.

Proof. In Algorithm 5, Line 1-2 runs Expand twice. In Expand, line 11-13 loops at most $\theta_V \cdot |U|$ times, and line 15-17 loops at most $|V|$ times. The operation to get maximum (minimum) value at line 8, 12 and 16 take $O(1)$ time by adapting the methods of decomposition algorithm [5]. As a result, Expand takes $O(\theta n)$ time, and the heuristic algorithm takes totally $O(\theta n + m)$ time. □

4.4 Parallelization

To further enhance the scalability of our algorithms for handling larger-scale graphs, we implement parallelization strategy based on our proposed algorithms and optimization techniques. The parallelization of our branch-and-bound and pivoting-based algorithms is based on the search tree of the algorithm (as shown in Fig. 2(b) and Fig. 3(b)). Specifically, for a given branching strategy, once an instance (S, C) corresponding to a branch is determined, the sub-branches generated by this branch are uniquely defined and mutually independent. Therefore, we parallelize the processing of sub-branches at lower levels of the search tree. Additionally, the parallelization process is controlled by a global counter that tracks the total number of parallel tasks, and when the counter reaches the number of limited threads, the sub-branches generated by each branch are processed sequentially. Additionally, we also perform parallelization on the ordering-based reduction through the traversal of ordered vertices, and on the common-neighbor-based reduction through the loop in lines 2-13 of Algorithm 3.

4.5 Optimized Algorithms

All the proposed optimization techniques can be applied to our branch-and-bound algorithm and pivoting-based algorithm. We refer to the branch-and-bound algorithm and pivoting-based algorithm equipped with all the optimization techniques as MDBB and MDBP, respectively. An implementation of MDBP is presented

Algorithm 5: Heuristic(G, k, θ)

```
   Input: Bipartite graph G = (U, V, E), integers k and θ
   Output: A large k-defective biclique
1  (U_0, V_0) ← Expand(U, V);
2  (V_1, U_1) ← Expand(V, U);
3  if |E_(U_0,V_0)| ≥ |E_(U_1,V_1)| then return G(U_0, V_0);
4  else return G(U_1, V_1);
5  Function Expand(U, V):
6      U' ← ∅; V' ← V;
7      while |U'| < θ do
8          u ← arg max_{u∈U\U'} d_{V'}(u);
9          if d_{V'}(u) + k < θ then break;
10         d̄ ← 0;
11         for i in 1 ... θ do
12             v ← arg min_{v∈V'} d̄_{U'∪{u}}(v);
13             d̄ ← d̄ + d̄_{U'∪{u}}(v);
14         if d̄ ≤ k then
15             while |Ē_(U',V')| + d̄_{V'}(u) > k do
16                 v ← arg max_{v∈V'} d̄_{U'∪{u}}(v);
17                 V' ← V' \ {v};
18             U' ← U' ∪ {u};
19     if |U'| < θ then return (∅, ∅);
20     else return (U', V');
```

Algorithm 6: MDBP

```
   Input: A bipartite graph G = (U, V, E), integers k, θ
   Output: A k-MDB of G
1  D* ← Heuristic(G, k, θ);
2  θ_U ← max_{u∈U} d(u);
3  while θ_U > θ do
4      θ_V ← max{θ, ⌊|D*|/θ_U⌋}; θ_U ← max{θ, ⌊θ_U/2⌋};
5      G' ← parallel CnReduction(G', k, min{θ_U, θ_V});
6      {u_1, u_2, ..., u_|U|} ← the descending degree order of U;
7      parallel for i ∈ 1 ... |U| do
8          S ← ({u_i}, ∅); C ← (N²₊(u_i), ⋃_{v∈N²₊(u_i)} N(v));
9          C ← Reduce(C, u_i, θ_U, θ_V);
10         BranchP*(S, C); // BranchP equipped with graph reduction
               techniques, upper-bounding techniques and parallelization
11 return D*;
12 Function Reduce(C = (U_C, V_C), u, θ_U, θ_V):
13     U_C' ← {v ∈ U_C | cn(u, v) ≥ θ_V − k};
14     V_C' ← {v ∈ V_C | d(v) ≥ θ_U − k};
15     return (U_C', V_C')
```

in Algorithm 6. The algorithm begins by obtaining an initial k-defective biclique (line 1) by performing the heuristic approach. Then it starts to find k-MDB by performing the progressive bounding reduction (lines 2-4). In each iteration, two size thresholds θ_U and θ_V are generated based on Eq. (6) and Eq. (7). Then, the parallelized common-neighbor-based reduction are employed on the input graph G (line 5) by invoking `CnReduction` (Algorithm 3). the algorithm continues by performing the parallelized ordering-based reduction on the reduced graph (lines 6-10). In detail, it traverses each vertex u of U in descending degree order (lines 7-8), and constructs an initial instance (S, C). After that, the algorithm further reduce the size of C by invoking `Reduce` (line 9), which utilizes the common-neighbor-based reduction (lines 13-14). Finally, the algorithm runs `BranchP`* to find k-MDB in (S, C), which is adapted from `BranchP` in Algorithm 2 by additionally implementing the graph reduction techniques and the upper-bounding techniques (Algorithm 4). By taking the maximum returned solution from all invocations of `BranchP`*, the algorithm derives a k-MDB D^* at the end and returns D^* as the final answer (line 11). The implementation of `MDBB` is quite similar to that of `MDBP`, which differs only in that replacing `BranchP`* in Algorithm 6 with `BranchB` from Algorithm 1 incorporating the optimization techniques used in `BranchP`*.

5 Experiments

5.1 Experimental Setup

Datasets. We collect 11 real-world bipartite graphs from diverse categories to evaluate the efficiency of our algorithms. These graphs have been widely used as benchmark datasets for evaluating the performance of other bipartite subgraph search problem [15, 37, 55]. The detailed information of these graphs are shown in Table 1, where Δ and $\overline{d}$ represent the maximum degree and average degree of vertices on each side, respectively. All the datasets are available at http://konect.cc/networks.

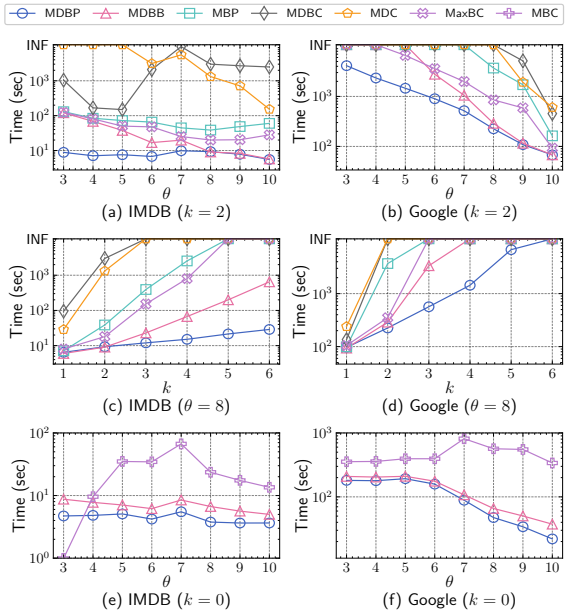
Algorithms. We implement two proposed algorithm `MDBB` and `MDBP`, which are respectively the branch-and-bound algorithm in Algorithm 1 and the pivoting-based algorithm in Algorithm 2 equipped with all our proposed optimization techniques in Sec. 4.

For comparative analysis, we implement 4 baseline algorithms adapted from other problems: `MDBC`, `MDC`, `MBP` and `MaxBC`. Specifically, `MDBC` is adapted from the state-of-the-art maximum vertex defective biclique algorithm `MDBC` [53] with the maximum-vertex-based reductions removed, as vertex maximization does not guarantee edge maximization. `MDC` is adapted from the maximum defective clique enumeration algorithm [13] by treating all vertices on the same side adjacent. `MBP` and `MaxBC` are adapted from the maximum biplex search algorithm [55] and maximal biclique enumeration algorithm [1], which are implemented by reconstructing their branch-and-bound and pivoting techniques to suit our problem. For fairness, these algorithms are incorporate the optimization techniques outlined in Sec. 4.1 and 4.3. All algorithms are implemented in C++ and executed on a PC equipped with a 2.2GHz CPU and 128GB RAM running Linux.

Parameter settings. During the evaluations of all algorithms, we vary the value of k from 1 to 6. As outlined in Sec. 2, we focus on finding a connected k-MDB, achieved by setting $\theta > k$. Based on this, we vary the value of θ from $k+1$ to 10 for a specified k.

Table 1: Datasets used in experiments

Dataset	$\lvert U\rvert$	$\lvert V\rvert$	$\lvert E\rvert$	Δ	$\overline{d}$
LKML	42,045	337,509	599,858	(31,719, 6,627)	(14, 1)
Team	901,130	34,461	1,366,466	(17, 2,671)	(1, 39)
Mummun	175,214	530,418	1,890,661	(968, 19805)	(10, 3)
Citeu	153,277	731,769	2,338,554	(189,292, 1,264)	(15, 3)
IMDB	303,617	896,302	3,782,463	(1,334, 1,590)	(12, 4)
Enwiki	18,038	2,190,091	4,129,231	(324,254, 1,127)	(228, 1)
Amazon	2,146,057	1,230,915	5,743,258	(12,180, 3,096)	(2, 4)
Twitter	244,537	9,129,669	10,214,177	(640, 18,874)	(41, 1)
Aol	4,811,647	1,632,788	10,741,953	(100,629, 84,530)	(2, 6)
Google	5,998,790	4,443,631	20,592,962	(423, 95,165)	(3, 4)
LiveJournal	3,201,203	7,489,073	112,307,385	(300, 1,053,676)	(35, 14)



(a) IMDB ($k = 2$) (b) Google ($k = 2$) (c) IMDB ($\theta = 8$) (d) Google ($\theta = 8$) (e) IMDB ($k = 0$) (f) Google ($k = 0$)

Figure 4: Runtime with varying θ and k

5.2 Performance Studies

Exp1: Efficiency comparison among different algorithms. We evaluate the efficiency of our algorithms, `MDBP` and `MDBB`, against four baselines: `MDBC`, `MDC`, `MBP`, and `MaxBC`, as shown in Table 2. Here, $|E^*|$ denotes the number of edges in k-MDB, and "-" indicates a timeout exceeding 3 hours. As observed, `MDBP` demonstrates superior performance. It consistently runs up to two orders of magnitude faster than `MDBC`, and successfully solves nearly all instances for $k \geq 3$ where baselines severely struggle with timeouts. For example, on the *LKML* dataset ($k = 3, \theta = 7$), `MDBP` completes in only 9.53s, yielding a $\approx 1000\times$ speedup over the fastest baseline `MaxBC`, which takes about 2.5 hours (9, 223s). Comparing our two proposed methods, `MDBP` and `MDBB` perform comparably when $k \leq 2$ and $\theta \geq 5$. However, `MDBP` exhibits a significant advantage when $k \geq 3$ or $\theta \leq 4$. For instance, on *Twitter* with $k = 4, \theta = 5$, `MDBB` takes nearly 1 hour (3, 550s) while `MDBP` finished in only 21s, achieving a speedup of over 160×. These results firmly underscore the effectiveness and high efficiency of the pivoting-based branching strategy introduced in Sec. 3.2. Furthermore, `MDBB` still consistently retains advantages over the four baseline algorithms, validating the utility of the new binary branching strategy detailed in Sec. 3.1.

Exp2: Runtime with varying θ and k. We evaluate `MDBB` and `MDBP`, along with 4 baseline algorithms, with varying θ and k on two representative datasets, *IMDB* and *Google*. The results are shown

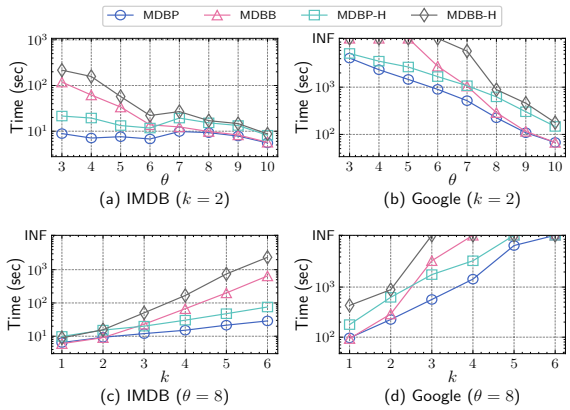
Table 2: Results of runtime among different algorithms on 10 real-world graphs (in seconds)

Dataset	k	θ	$\lvert E^*\rvert$	MDBP	MDBB	MDBC	MDC	MBP	MaxBC	k	θ	$\lvert E^*\rvert$	MDBP	MDBB	MDBC	MDC	MBP	MaxBC	k	θ	$\lvert E^*\rvert$	MDBP	MDBB	MDBC	MDC	MBP	MaxBC
LKML	1	2	1,781	**13**	17	-	-	42	229	2	3	460	**44**	-	-	-	-	-	3	4	149	**267**	-	-	-	-	-
		5	84	**1.61**	2.15	32	28	3.19	5.57		6	70	**7.60**	30	4,647	-	3,024	364		7	60	**9.53**	37	-	-	-	9,223
	4	7	66	**103**	1,216	-	-	-	-	5	8	67	**107**	1,224	-	-	-	-	6	9	0	**248**	1,732	-	-	-	-
		8	60	**9.36**	46	-	-	-	-		9	0	**13**	52	-	-	-	-		10	0	**12**	46	-	-	-	-
Team	1	2	701	**19**	39	-	-	48	240	2	3	265	**45**	-	-	-	-	1,783	3	4	149	**154**	-	-	-	-	-
		5	99	**7.14**	7.19	17	66	13	8.67		6	64	**9.53**	14	1,578	500	-	69		7	0	**10**	15	2,473	-	1,578	429
	4	5	111	**591**	-	-	-	-	-	5	6	79	**2,684**	-	-	-	-	-	6	8	0	**927**	-	-	-	-	-
		8	0	**9.92**	15	1,516	-	-	3,568		9	0	**10**	16	1,830	-	-	-		10	0	**7.65**	**11**	2,059	-	-	-
Mummun	1	2	10,315	**18**	19	-	-	36	286	2	3	8,803	**23**	1,831	-	-	8,519	1,309	3	4	4,185	**40**	-	-	-	-	-
		5	2,469	**13**	14	331	-	17	22		6	1,768	**21**	37	-	-	46	38		7	1,145	**33**	916	-	-	4,685	1,077
	4	5	2,481	**45**	-	-	-	-	-	5	6	1,783	**69**	-	-	-	-	-	6	7	1,163	**161**	-	-	-	-	-
		8	878	**38**	5,539	-	-	-	8,031		9	886	**41**	6,737	-	-	-	-		10	804	**53**	-	-	-	-	-
Citeu	1	2	12,425	**141**	147	-	-	1,796	3,962	2	3	12,427	**148**	188	-	-	1,841	-	3	4	9,989	**137**	4,333	-	-	-	9,183
		5	7,394	**5.48**	5.69	8,174	-	19	47		6	4,960	**7.88**	77	-	-	4,257	87		7	3,063	**18**	3,501	-	-	-	443
	4	5	7,406	**142**	-	-	-	-	-	5	6	4,975	**158**	-	-	-	-	-	6	7	3,081	**253**	-	-	-	-	-
		8	2,600	**26**	-	-	-	-	609		9	2,600	**36**	-	-	-	-	-		10	2,614	**134**	-	-	-	-	-
IMDB	1	2	775	5.99	**5.39**	447	-	17	60	2	3	562	**8.86**	17	1,041	-	30	126	3	4	522	**16**	436	7,924	-	-	158
		5	514	6.12	**5.35**	7.74	23	6.93	11		6	514	**6.76**	6.91	2,096	1,123	27	17		7	482	**15**	25	2,504	-	280	24
	4	5	526	**20**	594	-	-	-	267	5	6	529	**31**	684	-	-	-	901	6	7	498	**57**	7,936	-	-	-	-
		8	436	**15**	67	-	-	2,632	123		9	436	**19**	171	-	-	-	8,211		10	426	**23**	403	-	-	-	-
Enwiki	1	2	203,569	**3,630**	-	-	-	-	-	2	3	133,759	**4,978**	-	-	-	-	-	3	4	80,461	**3,260**	-	-	-	-	-
		5	21,939	**157**	159	965	387	378	457		6	4,240	**200**	583	-	-	-	-		7	3,224	**264**	-	-	-	-	3,382
	4	5	21,951	**3,826**	-	-	-	-	-	5	6	4,255	**5,744**	-	-	-	-	-	6	8	2,330	**6,558**	-	-	-	-	-
		8	2,316	**340**	-	-	-	-	-		9	1,696	**395**	-	-	-	-	-		10	1,274	**640**	-	-	-	-	-
Amazon	1	2	1,319	**43**	45	2,698	-	51	407	2	3	427	**57**	646	-	-	1,810	1,029	3	4	423	**89**	1,196	-	-	-	5,720
		5	413	12	**11**	14	50	16	17		6	418	13	**12**	38	-	20	29		7	357	**13**	15	1,184	-	629	155
	4	5	428	**100**	850	-	-	-	-	5	6	433	**250**	-	-	-	-	-	6	7	390	**268**	-	-	-	-	-
		8	368	**14**	16	-	-	-	-		9	379	**16**	20	-	-	-	-		10	390	**19**	26	-	-	-	-
Twitter	1	2	5,369	**11**	13	-	-	82	95	2	3	5,383	**11**	51	-	-	282	132	3	4	5,397	**15**	321	-	-	5,259	337
		5	5,369	**6.27**	7.60	1,229	8,976	15	33		6	5,383	**6.63**	18	6,693	-	87	64		7	5,397	**9.76**	54	-	-	751	158
	4	5	5,411	**21**	3,550	-	-	-	-	5	6	5,425	**31**	-	-	-	-	-	6	7	5,439	**58**	-	-	-	-	-
		8	5,411	**13**	292	-	-	-	597		9	5,425	**24**	3,268	-	-	-	6,206		10	5,439	**44**	9,895	-	-	-	-
Aol	1	2	10,869	1,033	**971**	-	-	1,427	-	2	3	3,208	**1,636**	-	-	-	-	-	3	4	853	**3,123**	-	-	-	-	-
		5	334	50	**49**	214	5,703	227	336		6	238	**94**	182	-	-	6,458	-		7	207	**123**	409	-	-	-	-
	4	5	346	**9,272**	-	-	-	-	-	5	7	219	**2,388**	-	-	-	-	-	6	9	174	-	-	-	-	-	-
		8	188	**470**	1,011	-	-	-	-		9	166	**4,497**	5,116	-	-	-	-		10	164	**5,585**	8,636	-	-	-	-
Google	1	2	10,829	**2,163**	2,526	-	-	2,582	-	2	3	3,370	**4,121**	-	-	-	-	-	3	4	1,201	**6,250**	-	-	-	-	-
		5	439	**275**	376	1,858	-	353	362		6	184	**898**	2,772	-	-	-	-		7	151	**1,351**	-	-	-	-	-
	4	7	157	**5,331**	-	-	-	-	-	5	8	155	**6,750**	-	-	-	-	-	6	9	156	**8,756**	-	-	-	-	-
		9	140	**613**	9,387	-	-	-	-		10	145	**684**	-	-	-	-	-		10	145	**1,978**	-	-	-	-	-

in Fig. 4, where the notion "INF" represents the runtime exceeds 3 hours. As observed, both MDBP and MDBB consistently outperform baseline algorithms, with MDBP achieving a speedup of 500×. For example, in Fig. 4(c), MDBP complete in 20s at $\theta = 7$, while the baselines cannot finish within 3 hours. When $\theta \geq 8$ or $k \leq 2$, MDBP and MDBB have similar runtimes, demonstrating the efficacy of our optimization techniques in reducing graph size. Furthermore, as k increases or θ decreases, the time curve of MDBP shows a gentler slope compared to other algorithms, indicating its lower sensitivity to input parameters and superior scalability. Additionally, we evaluate MDBB and MDBP in searching for the maximum biclique by setting $k = 0$. We compare them with a state-of-the-art maximum biclique search algorithm MBC [37], as shown in Fig. 4(e) and Fig. 4(f). As observed, MDBP and MDBB exhibit an advantage over MaxBC due to: (1) both of our algorithms achieves better time complexities than MaxBC, which has a time complexity of $O^*(2^n)$; and (2) our algorithms incorporate more advanced heuristic approach and graph reduction techniques, further enhancing their efficiency.

Exp3: Efficacy of the heuristic approach. To evaluate the effectiveness of our proposed heuristic approach, we compare MDBB and MDBP with their versions without the heuristic approach, referred to as MDBB-H and MDBP-H, respectively. The results are shown in Fig. 5. As observed, MDBB and MDBP always outperform MDBB-H and MDBP-H, respectively. This is because the heuristic approach provides a relatively large k-defective biclique at the beginning, which enhances the effectiveness of our upper-bounding techniques. Additionally, we observe that MDBP-H still always performs better than MDBB when $\theta \leq 6$ or $k \geq 3$, which further highlights the high efficiency of our pivoting techniques.

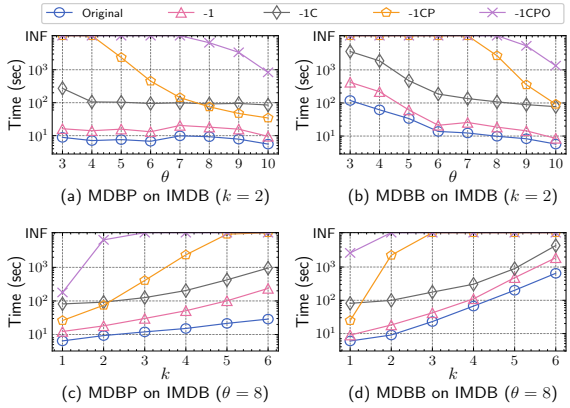
Exp4: Efficacy of graph reduction techniques. To evaluate the effectiveness of our graph reduction techniques, we implement



(a) IMDB ($k = 2$) (b) Google ($k = 2$)
(c) IMDB ($\theta = 8$) (d) Google ($\theta = 8$)

Figure 5: Runtime without using the heuristic approach

4 ablation versions of MDBP and MDBB by successively removing the following reductions: one-non-neighbor reduction, common-neighbor-based reduction, progressive bounding reduction and ordering-based reduction. The results are presented in Fig 6, which label the original algorithm as "Original", and the 4 ablation versions as "-1", "-1C", "-1CP", "-1CPO". As observed, all graph reduction techniques contribute to efficiency improvements. Specifically, the one-non-neighbor and common-neighbor-based reductions yield stable improvements with varying θ and k, while the progressive bounding and ordering-based reduction demonstrate more improvements when θ decreases or k increases. Notably, when $k = 1$ or $\theta = 10$, versions of MDBB and MDBP without progressive bounding reduction still outperform those with it. This suggests that our algorithms are well-designed and robust, achieving strong performance for small k or large θ even without progressive bounding reduction.



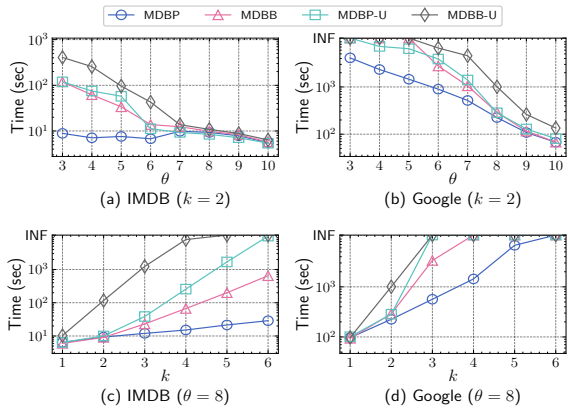
(a) MDBP on IMDB ($k = 2$) (b) MDBB on IMDB ($k = 2$)

(c) MDBP on IMDB ($\theta = 8$) (d) MDBB on IMDB ($\theta = 8$)

Figure 6: Runtime without graph reduction techniques

Table 3: Runtime on synthetic graphs (in seconds)

Distribution	ρ	Δ	$\overline{d}$	$k=1,\theta=2$			$k=3,\theta=4$			$k=5,\theta=6$		
				$\lvert E^*\rvert$	MDBP	MDBB	$\lvert E^*\rvert$	MDBP	MDBB	$\lvert E^*\rvert$	MDBP	MDBB
Power-law	0.2	(97, 82)	(19, 19)	197	**0.10**	0.18	181	**0.29**	0.96	175	**8.03**	11.82
	0.3	(77, 99)	(29, 29)	269	**0.11**	0.33	277	**0.56**	9.52	292	**16.02**	133.29
	0.4	(95, 100)	(36, 36)	349	**0.27**	**0.51**	357	**1.76**	26.01	375	**33.84**	432.58
	0.5	(100, 99)	(45, 45)	482	**0.92**	**1.94**	494	**5.83**	105.49	506	**189.92**	2,030.16
Normal	0.2	(29, 29)	(17, 17)	27	**0.26**	0.47	29	**31.74**	75.84	32	**59.74**	153.87
	0.3	(43, 43)	(29, 29)	44	**1.24**	3.04	45	**179.49**	397.23	49	**312.55**	1,225.26
	0.4	(48, 51)	(39, 39)	56	**3.78**	10.73	67	**489.87**	1,015.43	65	**1,300.01**	-
	0.5	(62, 61)	(49, 49)	65	**15.03**	43.81	65	**2,095.58**	-	-	-	-



(a) IMDB ($k = 2$) (b) Google ($k = 2$)

(c) IMDB ($\theta = 8$) (d) Google ($\theta = 8$)

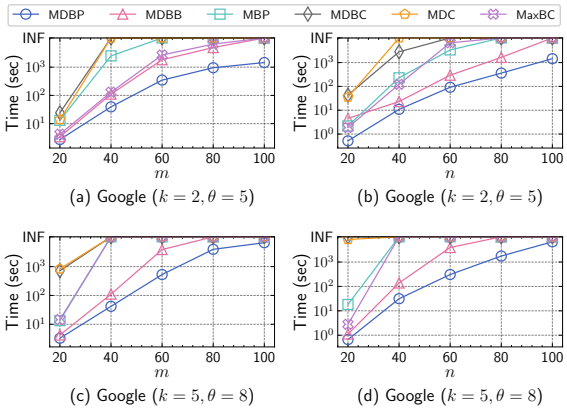
Figure 7: Runtime without upper-bounding techniques

Exp5: Efficacy of upper-bounding techniques. To evaluate the effectiveness of our proposed upper-bounding techniques, we compare MDBB and MDBP with their versions excluding the upper-bounding techniques (Algorithm 4), denoted as MDBB−H and MDBP−H, respectively. The results are shown in Fig. 7. As observed, incorporating the upper-bounding techniques significantly improves the efficiency of both MDBB and MDBP. Moreover, the improvement becomes more pronounced as k increases or θ decreases, indicating the strong effectiveness and efficiency of our proposed upper-bounding techniques. Additionally, the improvement of MDBP by performing upper-bounding techniques is more substantial compared to MDBB. This is likely because the pivoting-based branching strategy introduces more non-edges to the partial set S earlier, resulting in tighter vertex and edge upper bounds derived from the upper-bounding techniques, which helps prune more unnecessary instances.

Exp6: Performance on synthetic graphs. To discover the relationship between algorithm efficiency and graph properties, we evaluate MDBB and MDBP on synthetic graphs with varying vertex degree distributions and graph densities. Specifically, we generate 8 synthetic graphs with 100 vertices per side, using either power-law or normal degree distribution, with density $\rho = \frac{|E|}{|U|\cdot|V|}$ varies from 0.2 to 0.5. The results are presented in Table 3. As observed, our algorithms consistently exhibits high efficiency on power-law distribution graphs, while performance on normal distribution graphs is more significantly affected by ρ and k. This can be attributed to two key factors: First, power-law graphs exhibit a larger gap between Δ and $\overline{d}$, which enhances the effectiveness of our branching strategies and graph reduction techniques. This explains why our approaches are highly efficient on large-scale real-world graphs with a significantly gap between Δ and $\overline{d}$, as illustrated in Table 1 and 2. Second, the higher number of k-MDB edges in power-law graphs benefits our upper bound techniques, which can also reflected in the real-world graphs in Table 2. Additionally, We observe that MDBP outperforms MDBB on these synthetic graphs, indicating the effectiveness of our pivoting-based branching strategy.

Table 4: Runtime of parallelized algorithms (in seconds)

Dataset	k	θ	MDBP with threads				MDBB with threads			
			1	10	20	30	1	10	20	30
Google	1	2	2,163	425	56	54	3,052	870	223	224
	3	4	6,250	1,324	722	795	-	-	10,649	10,413
	5	6	-	-	8,947	9,062	-	-	-	-
LiveJournal	1	2	119	89	47	44	-	-	9,314	9,335
	3	4	3,996	997	850	845	-	-	-	-
	5	6	-	-	10,016	9,943	-	-	-	-



(a) Google ($k = 2, \theta = 5$) (b) Google ($k = 2, \theta = 5$)

(c) Google ($k = 5, \theta = 8$) (d) Google ($k = 5, \theta = 8$)

Figure 8: Scalability of different algorithms

Exp7: Scalability testing. To evaluate the scalability of algorithms, we test the runtime of MDBB, MDBP, MDBC, MDC, MBP, MaxBC on *Google* using random sampling of vertices or edges. Results on the other datasets are consistent. We prepare 8 proportionally sampled subgraphs of *Google* by sampling 20%, 40%, 60% and 80% vertices or edges from the original graph. The results are shown in Fig. 8. As observed, MDBP shows the slowest runtime growth as the graph size increases, while MDBC and MDC exhibit the steepest rise. This result indicates that MDBP achieves the best scalability among all the tested algorithms. Furthermore, regardless of graph size, both MDBP and MDBB significantly outperform other algorithms, indicating the advancement of our approach.

Exp8: Efficiency of parallelized algorithms. We evaluate the efficiency of parallel MDBB and MDBP on two challenging large-scale graphs *Google* and *LiveJournal*, with varying the limitation of threads as 1, 10, 20 and 30, which is also the threshold of global counter. The results are shown in Table 4. As observed, the runtime of MDBP and MDBB consistently decreases up to 20 threads,

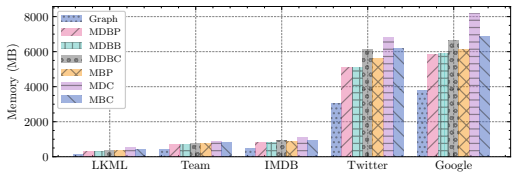
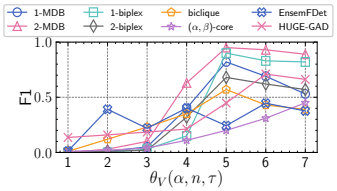


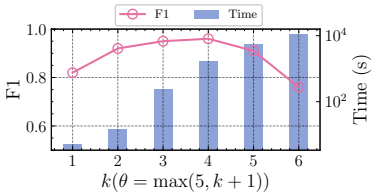
Figure 9: Memory usage of various algorithms



(a) F1-score with varying parameters

Method	F1	Modularity	Time(s)
1-MDB	0.82	0.33	0.30
2-MDB	**0.95**	**0.45**	15.44
1-biplex	0.92	0.37	35.15
2-biplex	0.78	0.24	1187.82
biclique	0.57	0.15	0.23
(α, β)-core	0.45	0.04	0.14
EnsemFDet	0.45	0.17	59.60
HUGE-GAD	0.71	0.20	882.67

(b) Optimal performance



(c) Performance of k-MDB with varying k

Figure 10: Performance on fraud detection task

achieving up to 40× speedup on *Google* ($k = 1, \theta = 2$) and 10× on *LiveJournal* ($k = 5, \theta = 6$). The runtime stabilize between 20 threads and 30 threads, suggesting that running MDBP and MDBB under 20 threads yields the best performance. Notably, the parallelization effectively handles complex settings ($k = 5, \theta = 6$) that cause the sequential versions to time out after 3 hours, which demonstrates the advancement of our parallelization technique.

Exp9: Memory usage. We evaluate the maximum physical memory usage (in MB) of the algorithms MDBB, MDBP, MDBC, MBP, MDC and MaxBC on five representative graphs, with the parameters $k = 2$ and $\theta = 3$. Similar trends hold for other datasets and parameters. The results are shown in Fig. 9. As observed, all algorithms except MDC and MaxBC consumes less than twice the original graph's memory. This is mainly because MDC adds massive intra-side edges, and MaxBC maintains an extra DAG structure during pivoting. These results indicate that our proposed algorithms are space efficient.

5.3 Case Study

Case 1: Fraud Detection. We evaluate k-MDB in a fraud detection task using a simulated camouflage attack, following the experiment described in [25]. We simulate the camouflage attack on the *Appliance* dataset (available at https://amazon-reviews-2023.github.io/), which contains 602,777 reviews on 30,459 products by 515,650 users. We inject a fraud block consisting of 500 fake users, 500 fake products, along with 10,000 fake reviews by randomly connecting fake users and fake products, and 10,000 camouflage reviews by randomly connecting fake users and real products. By slightly modifying MDBP using the similar method in [55], we identify the top 2000 k-MDBs with the algorithm MDBP to flag fake items.

We compared our approach against maximum biclique [37], maximum k-biplex [55], (α, β)-core [7], alone with a densest-subgraph-based method EnsemFDet [43] and a GNN-based method HUGE-GAD [40]. For EnsemFDet, we report the better performance achieved

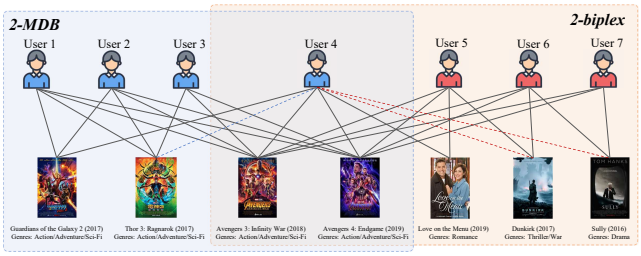


Figure 11: Recommendation results on *MovieLens*

when accelerated by either the Spade [26] or Dupin [27] parallel framework. Let θ_U and θ_V denote the user and product size thresholds constraining k-MDB, maximum k-biplex, and maximum biclique. Let n denote the sampled block count for EnsemFDet, and τ the minimum shared products required to connect users in HUGE-GAD's attributed graph; both are key parameters impacting detection accuracy. We evaluated F1-scores ($\frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision}+\text{recall}}$) by varying θ_V, α, n and τ from 1 to 7 and fixing $\theta_U = \beta = 4$. As shown in Fig. 10(a), 2-MDB consistently outperforms others when $\theta_V \geq 5$. Additionally, the F1-score of 2-MDB rises over 1-MDB while 2-biplex drops below 1-biplex, as k-MDB offers finer-grained density relaxation than k-biplex. For $\theta_V \leq 3$, biclique, EnsemFDet and HUGE-GAD outperform k-MDB and k-biplex, as the latter two are too sparse to provide meaningful structure. When $\theta_V \geq 5$, the F1-score of biclique declines due to its strict definition limiting recall. In contrast, the more relaxed k-MDB and k-biplex still maintain high F1-scores.

We also assessed running time and subgraph quality using the widely adopted modularity ($\frac{1}{2m} \sum_{u \in U_S, v \in V_S} (A_{uv} - \frac{d(u)d(v)}{2m})$), which quantifies the strength of internal connectivity relative to a random distribution and serves as a key indicator for assessing the community structure within the subgraph. Fig. 10(b) demonstrates that 2-MDB achieves the highest modularity, indicating superior community structure that drives its high F1-score. Although 1-biplex achieves a comparable F1-score, its running time is significantly higher. These results confirm the robust effectiveness and efficiency of our solutions for real-world fraud detection applications.

To determine the optimal value of k, we evaluate the F1-score and running time of k-MDB with varying k from 1 to 6, as shown in Fig.10(c). Since k-MDB yields the best results when $\theta = 5$ according to Fig. 10(a), we set $\theta = \max(5, k + 1)$ to ensure optimality. At $k = 4$, k-MDB achieves peak accuracy (F1 score = 0.97) but requires a lengthy execution time ($\approx$ 30 minutes), making it suitable for offline fraud detection. Conversely, configuring $k = 2$ results in a slight accuracy drop (F1 score = 0.95) alongside a significant efficiency boost ($\approx$ 15 seconds), which is ideal for real-time detection scenarios.

Case 2: Online Recommendation. We conduct another case study to evaluate k-MDB in an online recommendation task. We use the *MovieLens* dataset (available at https://grouplens.org/datasets/movielens/32m/), which contains 32 million ratings applied to 87,585 movies by 200,948 users. To evaluate online recommendations, we restrict the dataset to movies released in 2016 or later and construct a bipartite graph where edges represent 5.0 ratings. We leverage the non-edges within the extracted 2-MDB and maximum 2-biplex subgraphs to formulate recommendations, as illustrated in Fig. 11. Specifically, the 2-MDB model identifies a cohesive user community centered around "Action/Adventure/Sci-Fi" genres. Guided by its non-edges, it accurately recommends a genre-matched movie,

"Thor: Ragnarok", to User 4 (blue dashed line). In contrast, the maximum 2-biplex yields an anomalous recommendation, erroneously suggesting "Dunkirk" and "Sully"—two films from completely different genres—to User 4 (red dashed line). This discrepancy is primarily caused by the structural sparsity inherent in the 2-biplex model. Its degree-based relaxation permits a user group bound only by shared interactions with mainstream blockbusters (e.g., The Avengers), which consequently introduces unrelated movies into the sparse subgraph's recommendations. Conversely, the strict global density of the 2-MDB model guarantees the detection of cohesive communities, precisely pinpointing a small subset of unobserved, genre-consistent movies for each target user.

6 Conclusion

In this paper, we investigate the problem of finding a maximum k-defective biclique in a bipartite graph. We propose two novel algorithms to tackle the problem. The first algorithm employs a newly-designed binary branching strategy that reduces the time complexity to below $O^*(2^n)$, while the second algorithm utilizes a novel pivoting-based branching strategy, achieving a notably lower time complexity. Additionally, we propose a series of non-trivial optimization techniques to further improve the efficiency of our algorithms. Extensive experiments conducted on 10 real-word graphs demonstrate the notable effectiveness and efficiency of our algorithms and optimization techniques, and two case studies validate the effectiveness of maximum k-defective biclique model in real-world applications.

References

[1] Aman Abidi, Rui Zhou, Lu Chen, and Chengfei Liu. 2020. Pivot-based Maximal Biclique Enumeration. In *IJCAI*. 3558–3564.

[2] Gabriela Alexe, Sorin Alexe, Yves Crama, Stephan Foldes, Peter L Hammer, and Bruno Simeone. 2004. Consensus algorithms for the generation of all maximal bicliques. *Discrete Applied Mathematics* 145, 1 (2004), 11–21.

[3] Taher Alzahrani and Kathy Horadam. 2019. Finding maximal bicliques in bipartite networks using node similarity. *Applied Network Science* 4, 1 (2019), 21.

[4] Christoph Ambühl, Monaldo Mastrolilli, and Ola Svensson. 2011. Inapproximability results for maximum edge biclique, minimum linear arrangement, and sparsest cut. *SIAM J. Comput.* 40, 2 (2011), 567–596.

[5] Vladimir Batagelj and Matjaz Zaversnik. 2003. An o (m) algorithm for cores decomposition of networks. *arXiv preprint cs/0310049* (2003).

[6] Fabian Braun, Olivier Caelen, Evgueni N. Smirnov, Steven Kelk, and Bertrand Lebichot. 2017. Improving Card Fraud Detection Through Suspicious Pattern Discovery. In *Advances in Artificial Intelligence: From Theory to Practice*, Salem Benferhat, Karim Tabia, and Moonis Ali (Eds.). Springer International Publishing, 181–190.

[7] Monika Cerinšek and Vladimir Batagelj. 2015. Generalized two-mode cores. *Social Networks* 42 (2015), 80–87.

[8] Lijun Chang. 2023. Efficient Maximum k-Defective Clique Computation with Improved Time Complexity. *Proceedings of the ACM on Management of Data* 1, 3 (2023), 1–26.

[9] Lijun Chang. 2024. Maximum Defective Clique Computation: Improved Time Complexities and Practical Performance. *arXiv preprint arXiv:2403.07561* (2024).

[10] Lu Chen, Chengfei Liu, Rui Zhou, Jiajie Xu, and Jianxin Li. 2021. Efficient exact algorithms for maximum balanced biclique search in bipartite graphs. In *Proceedings of the 2021 International Conference on Management of Data*. 248–260.

[11] Lu Chen, Chengfei Liu, Rui Zhou, Jiajie Xu, and Jianxin Li. 2022. Efficient maximal biclique enumeration for large sparse bipartite graphs. *Proceedings of the VLDB Endowment* 15, 8 (2022), 1559–1571.

[12] Xiaoyu Chen, Yi Zhou, Jin-Kao Hao, and Mingyu Xiao. 2021. Computing maximum k-defective cliques in massive graphs. *Computers & Operations Research* 127 (2021), 105131.

[13] Qiangqiang Dai, Ronghua Li, Donghang Cui, and Guoren Wang. 2024. Theoretically and Practically Efficient Maximum Defective Clique Search. *Proceedings of the ACM on Management of Data* 2, 4 (2024), 1–27.

[14] Qiangqiang Dai, Rong-Hua Li, Donghang Cui, Meihao Liao, Yu-Xuan Qiu, and Guoren Wang. 2024. Efficient Maximal Biplex Enumerations with Improved Worst-Case Time Guarantee. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–26.

[15] Qiangqiang Dai, Rong-Hua Li, Xiaowei Ye, Meihao Liao, Weipeng Zhang, and Guoren Wang. 2023. Hereditary Cohesive Subgraphs Enumeration on Bipartite Graphs: The Power of Pivot-based Approaches. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–26.

[16] Milind Dawande, Pinar Keskinocak, Jayashankar M. Swaminathan, and Sridhar R. Tayur. 2001. On Bipartite and Multipartite Clique Problems. *J. Algorithms* 41, 2 (2001), 388–403.

[17] Kemal Eren, Mehmet Deveci, Onur Küçüktunç, and Ümit V Çatalyürek. 2013. A comparative analysis of biclustering algorithms for gene expression data. *Briefings in bioinformatics* 14, 3 (2013), 279–292.

[18] Qilong Feng, Shaohua Li, Zeyang Zhou, and Jianxin Wang. 2018. Parameterized algorithms for edge biclique and related problems. *Theoretical Computer Science* 734 (2018), 105–118.

[19] Fedor V Fomin and Petteri Kaski. 2013. Exact exponential algorithms. *Commun. ACM* 56, 3 (2013), 80–88.

[20] Jian Gao, Zhenghang Xu, Ruizhi Li, and Minghao Yin. 2022. An exact algorithm with new upper bounds for the maximum k-defective clique problem in massive sparse graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 10174–10183.

[21] Michael R Garey and David S Johnson. 1979. *Computers and intractability*. Vol. 174. freeman San Francisco.

[22] Nicolas Gillis and François Glineur. 2014. A continuous characterization of the maximum-edge biclique problem. *Journal of Global Optimization* 58, 3 (2014), 439–464.

[23] Timo Gschwind, Stefan Irnich, and Isabel Podlinski. 2018. Maximum weight relaxed cliques and Russian Doll Search revisited. *Discrete Applied Mathematics* 234 (2018), 131–138.

[24] Bryan Hooi, Kijung Shin, Hyun Ah Song, Alex Beutel, Neil Shah, and Christos Faloutsos. 2017. Graph-Based Fraud Detection in the Face of Camouflage. *ACM Trans. Knowl. Discov. Data* 11, 4 (2017), Article 44.

[25] Bryan Hooi, Hyun Ah Song, Alex Beutel, Neil Shah, Kijung Shin, and Christos Faloutsos. 2016. Fraudar: Bounding graph fraud in the face of camouflage. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 895–904.

[26] Jiaxin Jiang, Yuan Li, Bingsheng He, Bryan Hooi, Jia Chen, and Johan Kok Zhi Kang. 2022. Spade: A real-time fraud detection framework on evolving graphs. *Proceedings of the VLDB Endowment* 16, 3 (2022), 461–469.

[27] Jiaxin Jiang, Siyuan Yao, Yuchen Li, Qiange Wang, Bingsheng He, and Min Chen. 2025. Dupin: A parallel framework for densest subgraph discovery in fraud detection on massive graphs. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.

[28] Mingming Jin, Jiongzhi Zheng, and Kun He. 2024. KD-Club: An Efficient Exact Algorithm with New Coloring-based Upper Bound for the Maximum k-Defective Clique Problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 20735–20742.

[29] MA Langston, Elissa J Chesler, and Y Zhang. 2008. On finding bicliques in bipartite graphs: a novel algorithm with application to the integration of diverse biological data types. In *Proceedings of the 41st Annual Hawaii International Conference on System Sciences (HICSS 2008)(HICSS)*, Vol. 1. 473.

[30] Sune Lehmann, Martin Schwartz, and Lars Kai Hansen. 2008. Biclique communities. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 78, 1 (2008), 016108.

[31] Michael Ley. 2002. The DBLP Computer Science Bibliography: Evolution, Research Issues, Perspectives *(String Processing and Information Retrieval)*. Springer Berlin Heidelberg, 1–10.

[32] Jingdong Li, Zhao Li, Xiaoling Wang, Xingjian Lu, Ji Zhang, and Hongyang Chen. 2023. GPU-Accelerated Maximal Bicliques Mining Framework for Large E-commerce Networks. 539-544 pages.

[33] Jinyan Li, Guimei Liu, Haiquan Li, and Limsoon Wong. 2007. Maximal biclique subgraphs and closed pattern pairs of the adjacency matrix: A one-to-one correspondence and mining algorithms. *IEEE Transactions on Knowledge and Data Engineering* 19, 12 (2007), 1625–1637.

[34] Jinyan Li, Kelvin Sim, Guimei Liu, and Limsoon Wong. 2008. Maximal quasi-bicliques with balanced noise tolerance: Concepts and co-clustering applications. In *Proceedings of the 2008 SIAM International Conference on Data Mining*. SIAM, 72–83.

[35] Guimei Liu, Kelvin Sim, and Jinyan Li. 2006. Efficient mining of large maximal bicliques. In *Data Warehousing and Knowledge Discovery: 8th International Conference, DaWaK 2006, Krakow, Poland, September 4-8, 2006. Proceedings 8*. Springer, 437–448.

[36] Chunyu Luo, Yi Zhou Zhengren Wang, and Mingyu Xiao. 2024. A Faster Branching Algorithm for the Maximum k-Defective Clique Problem. *arXiv preprint arXiv:2407.16588* (2024).

[37] Bingqing Lyu, Lu Qin, Xuemin Lin, Ying Zhang, Zhengping Qian, and Jingren Zhou. 2020. Maximum biclique search at billion scale. *Proceedings of the VLDB Endowment* (2020).

[38] Sara C Madeira and Arlindo L Oliveira. 2004. Biclustering algorithms for biological data analysis: a survey. *IEEE/ACM transactions on computational biology and bioinformatics* 1, 1 (2004), 24–45.

[39] Pasin Manurangsi. 2018. Inapproximability of Maximum Biclique Problems, Minimum k-Cut and Densest At-Least-k-Subgraph from the Small Set Expansion Hypothesis. *Algorithms* 11, 1 (2018), 10.

[40] Junjun Pan, Yixin Liu, Xin Zheng, Yizhen Zheng, Alan Wee-Chung Liew, Fuyi Li, and Shirui Pan. [n. d.]. A label-free heterophily-guided approach for unsupervised graph fraud detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 12443–12451.

[41] René Peeters. 2003. The maximum edge biclique problem is NP-complete. *Discrete Applied Mathematics* 131, 3 (2003), 651–654.

[42] Amela Prelić, Stefan Bleuler, Philip Zimmermann, Anja Wille, Peter Bühlmann, Wilhelm Gruissem, Lars Hennig, Lothar Thiele, and Eckart Zitzler. 2006. A systematic comparison and evaluation of biclustering methods for gene expression data. *Bioinformatics* 22, 9 (2006), 1122–1129.

[43] Yuxiang Ren, Hao Zhu, Jiawei Zhang, Peng Dai, and Liefeng Bo. [n. d.]. Ensemfdet: An ensemble approach to fraud detection based on bipartite graph. In *2021 IEEE 37th international conference on data engineering (ICDE)*. IEEE, 2039–2044.

[44] Eran Shaham, Honghai Yu, and Xiao-Li Li. 2016. On finding the maximum edge biclique in a bipartite graph: a subspace clustering approach. In *Proceedings of the 2016 SIAM International Conference on Data Mining*. SIAM, 315–323.

[45] Shahram Shahinpour, Shirin Shirvani, Zeynep Ertem, and Sergiy Butenko. 2017. Scale reduction techniques for computing maximum induced bicliques. *Algorithms* 10, 4 (2017), 113.

[46] Vladimir Stozhkov, Austin Buchanan, Sergiy Butenko, and Vladimir Boginski. 2022. Continuous cubic formulations for cluster detection problems in networks. *Mathematical Programming* 196, 1 (2022), 279–307.

[47] Melih Sözdinler and Can Özturan. 2018. Finding maximum edge biclique in bipartite networks by integer programming. In *2018 IEEE International Conference on Computational Science and Engineering (CSE)*. IEEE, 132–137.

[48] Svyatoslav Trukhanov, Chitra Balasubramaniam, Balabhaskar Balasundaram, and Sergiy Butenko. 2013. Algorithms for detecting optimal hereditary structures in graphs, with application to clique relaxations. *Computational Optimization and Applications* 56 (2013), 113–130.

[49] Jianhua Wang, Jianye Yang, Ziyi Ma, Chengyuan Zhang, Shiyu Yang, and Wenjie Zhang. 2023. Efficient Maximal Biclique Enumeration on Large Uncertain Bipartite Graphs. *IEEE Trans. on Knowl. and Data Eng.* 35, 12 (2023), 12634–12648. doi:10.1109/tkde.2023.3272110

[50] Kai Wang, Yiheng Hu, Xuemin Lin, Wenjie Zhang, Lu Qin, and Ying Zhang. 2021. A Cohesive Structure Based Bipartite Graph Analytics System. 4799–4803 pages.

[51] Kai Wang, Wenjie Zhang, Xuemin Lin, Lu Qin, and Alexander Zhou. 2022. Efficient personalized maximum biclique search. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 498–511.

[52] Yiyuan Wang, Shaowei Cai, and Minghao Yin. 2018. New heuristic approaches for maximum balanced biclique problem. *Information Sciences* 432 (2018), 362–375.

[53] Z. Wang, L. Chang, and J. X. Yu. [n. d.]. Identifying Maximum Defective Bicliques in Large Bipartite Graphs. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 3710–3723. doi:10.1109/ICDE65448.2025.00277

[54] Haiyuan Yu, Alberto Paccanaro, Valery Trifonov, and Mark Gerstein. 2006. Predicting interactions in protein networks by completing defective cliques. *Bioinformatics* 22, 7 (2006), 823–829.

[55] Kaiqiang Yu and Cheng Long. 2023. Maximum k-Biplex Search on Bipartite Graphs: A Symmetric-BK Branching Approach. *Proc. ACM Manag. Data* 1, 1 (2023), Article 49.

[56] Kaiqiang Yu, Cheng Long, Shengxin Liu, and Da Yan. 2022. Efficient Algorithms for Maximal k-Biplex Enumeration. In *Proceedings of the 2022 International Conference on Management of Data*. 860–873.

[57] Bo Yuan, Bin Li, Huanhuan Chen, and Xin Yao. 2015. A New Evolutionary Algorithm with Structure Mutation for the Maximum Balanced Biclique Problem. *IEEE Transactions on Cybernetics* 45, 5 (2015), 1054–1067.

[58] Hongru Zhou, Shengxin Liu, and Ruidi Cao. 2025. Efficient Maximum Vertex (k,l)-Biplex Computation on Bipartite Graphs. *Tsinghua Science and Technology* 30, 2 (2025), 569–584. doi:10.26599/TST.2024.9010009

[59] Yi Zhou and Jin-Kao Hao. 2019. Tabu search with graph reduction for finding maximum balanced bicliques in bipartite graphs. *Eng. Appl. Artif. Intell.* 77 (2019), 86–97.